arXiv:2408.02920v2 [cs.SE] 9 Sep 2026

ANALYSIS OF ARCHITECTURE OPTIONS FOR FOUNDATION-MODEL-BASED AGENTS: A TAXONOMY AND DECISION MODEL

A PREPRINT

Jingwen Zhou[1,2], Qinghua Lu[2,3], Jieshan Chen[2], Liming Zhu[2,3], Xiwei Xu[2], Zhenchang Xing[2,4], and Stefan Harrer[2]

[1]School of Computer, Data and Mathematical Sciences, Western Sydney University, Sydney, Australia
[2]CSIRO's Data61, Australia
[3]University of New South Wales, Sydney, New South Wales, Australia
[4]Australian National University, Canberra, Australian Capital Territory, Australia

ABSTRACT

The rapid advancement of foundation models has led to a new generation of AI-based agents. While mature frameworks exist, the system-level architectural decision-making is still often ad-hoc, as architects lack a systematic, evidence-based framework for navigating the complex landscape of architectural options and their resulting trade-offs. To address this critical gap, we conducted a Systematic Literature Review (SLR) of 66 primary studies to develop two key artifacts: (1) a comprehensive taxonomy that systematically classifies architectural design options for foundation-model-based agents across functional capabilities and non-functional qualities, and (2) an accompanying decision model, presented as a matrix, that maps these options to their impact on system qualities, making critical design trade-offs explicit. The taxonomy and decision model were then validated through a multi-faceted process, including a formal survey with 12 experts from both academia and industry. This work provides architects with a common vocabulary and a structured guide for designing and reasoning about agent systems, thereby providing a rigorous foundation to address the current fragmentation in the field.

Keywords Foundation Model · Large Language Model · LLM Agent · Software Architecture · Taxonomy

1 Introduction

The emergence of Foundation Models (FMs)—large-scale models pre-trained on vast amounts of data represents a fundamental paradigm shift in Artificial Intelligence [1]. Their ability to adapt to a wide range of downstream tasks has enabled a new class of sophisticated applications, chief among them being Foundation-Model-based agents (FM-based agents). These are autonomous systems that leverage FMs as their core cognitive engine to perceive, reason, plan, and act to achieve complex goals, with the potential to revolutionize industries from healthcare to smart manufacturing [1–3]. The rapid proliferation of these agents, exemplified by systems like Auto-GPT[1] and BabyAGI[2] for autonomous task management, Voyager for embodied learning [4], UI-based computational agents like the OpenAI Operator[3], Claude

Email addresses: helen.zhou@data61.csiro.au (Jingwen Zhou), qinghua.lu@data61.csiro.au (Qinghua Lu), jieshan.chen@data61.csiro.au (Jieshan Chen), liming.zhu@data61.csiro.au (Liming Zhu), xiwei.xu@data61.csiro.au (Xiwei Xu), zhenchang.xing@data61.csiro.au (Zhenchang Xing), stefan.harrer@data61.csiro.au (Stefan Harrer)

[1]https://github.com/Significant-Gravitas/AutoGPT

[2]https://github.com/yoheinakajima/babyagi

[3]https://openai.com/index/introducing-operator/

Opus 5[4] that interact with computer systems through natural language interfaces, the first AI software engineer Devin[5], and open-source local execution tools like Open Interpreter[6], signals a move towards more capable and independent AI systems.

1.1 Motivation

However, this increasing capability brings significant architectural challenges. Today's agents employ cutting-edge technologies" and "new operational paradigms" that stretch the boundaries of traditional software design [1]. For instance, multi-agent frameworks like AutoGen [5] introduce complex architectural challenges in coordination of conversational workflows, ensuring effective communication protocols between agents, and maintaining a consistent state as multiple agents collaboratively attempt to complete a task. At the same time, agents like Voyager possess novel capabilities such as long-term *memory* and *reflection*, creating new design problems related to efficiency, and observability [4]. While structured frameworks like LangChain and Microsoft Semantic Kernel provide strong operational foundations, the higher-level architectural decision-making process remains fragmented. Consequently, system-level design choices are often ad-hoc, leaving architects without a common vocabulary and a systematic framework to reason about the trade-offs of their choices.

To fully appreciate these challenges, it is crucial to clarify what constitutes the **"architectures and frameworks"** in this context. An agent's **architecture** refers to the fundamental components that constitute the system (e.g., planning, memory, action modules) and their relationships. The **frameworks** that have emerged to build these systems, such as the aforementioned AutoGen [5], LangChain[7], data-centric solutions like LlamaIndex[8], orchestration frameworks like CrewAI[9], autonomous systems like SuperAGI[10], and platform-integrated solutions like the Microsoft Copilot Framework[11], provide reusable components and abstractions that expose this vast and complex design space to developers.

The proliferation of these powerful frameworks and the novel architectural patterns they enable has led to a fragmented landscape of ad-hoc designs. Architects lack a common vocabulary and a systematic method to reason about the trade-offs of their choices. This gap in shared architectural knowledge directly hinders the **"optimization of an architecture for improved functionality,"** which is the process of making conscious design choices to better achieve desired system behaviors (like autonomous learning) and quality attributes (like reliability). Without a clear understanding of the trade-offs for different architectural options, it is difficult to analyze and ensure these critical system-level qualities. A comprehensive taxonomy is therefore needed to provide the structured, evidence-based foundation for designing and analyzing these complex systems.

1.2 Scope and Contributions

To address this architectural gap, we introduce a comprehensive, evidence-based taxonomy of 74 architectural design options for FM-based agents, derived from a systematic literature review (SLR) of 66 primary studies. In this paper, we define an **architecture option** as a specific, fundamental design approach or pattern adopted to address a particular architectural concern (e.g., selecting a 'Centralized' vs. 'Fully Distributed' coordination mechanism). It is a more granular and concrete choice than a general "design decision" [6], as it refers to a selection from a predefined set of architectural alternatives within our taxonomy. Paired with this taxonomy, we introduce a decision model to analyze the trade-offs between these options, offering deeper insights and guiding the design of the next generation of FM-based agents. The main contributions of this paper are as follows:

- **A Comprehensive Taxonomy of Architectural Options:** We present a detailed, evidence-based taxonomy for FM-based agents, derived from a systematic literature review and refined through expert validation. The taxonomy systematically classifies architectural options across functional capabilities and non-functional qualities.

[4] https://www.anthropic.com/news/claude-opus-5

[5] https://devin.ai/

[6] https://www.openinterpreter.com/

[7] https://www.langchain.com/

[8] https://www.llamaindex.ai/

[9] https://www.crewai.com/

[10] https://superagi.com/

[11] https://news.microsoft.com/source/features/ai/microsoft-outlines-framework-for-building-ai-apps-and-copilots-expands-ai-plugin-ecosystem/

- **An Architectural Decision Model:** We introduce a practical decision model, presented as a Decision Matrix, that analyzes the trade-offs between architectural choices and their impact on key system qualities to guide architects in their design process.
- **Validation and discussion of the taxonomy's implications:** The taxonomy and decision model underwent a multi-faceted validation process, including a formal survey with 12 experts from both academia and industry. Furthermore, we articulate the taxonomy and decision model's specific implications for key stakeholders, including researchers, architects, and tool builders, clarifying its value for both academic research and industrial practice.

Specifically, Section 2 reviews related work. Section 3 details our systematic methodology. The complete taxonomy is presented in Section 4, followed by the decision model in Section 5. We describe our validation of these artifacts in Section 6. We discuss threats to validity in Section 7, and introduce the implications for key stakeholders in Section 8. Finally, Section 9 concludes the paper, summarizing our key findings and outlining future research directions.

2 Background and Related Work

2.1 FM-based Agent Frameworks

The landscape of FM-based agents is characterized by rapid advancements in both the foundational models that power them and the frameworks designed to orchestrate their behavior. Major technology companies have developed increasingly powerful Large Language Models (LLMs) that serve as the core cognitive engines for modern agents. For instance, Google's Gemini 3.6 Flash is optimised for reasoning, coding, and agentic tasks, and has been integrated across their products [7]. Similarly, Meta's Llama 4 represents another highly efficient and scalable foundation model, adopting a mixture-of-experts architecture to support long-context and multimodal tasks [8].

Building upon these powerful models, a rich ecosystem of agent frameworks has emerged to tackle complex, multi-step tasks. A significant focus has been on multi-agent collaboration. Microsoft's **AutoGen** is a prominent open-source framework designed to facilitate complex conversations and cooperation between multiple AI agents [5, 9]. **MetaGPT** takes a novel approach by emulating a software company's structure, assigning roles like project managers and engineers to different agents to collaboratively develop code [10]. **CrewAI** also focuses on enhancing multi-agent collaboration, enabling agents to work together to solve tasks [11]. Distinct from these, frameworks like **LangChain** provide a general-purpose toolkit for developers, focusing on integrating LLMs with a wide array of external tools and data sources to enable the seamless execution of complex workflows.

2.2 Existing Surveys and Positioning of Our Work

Despite the proliferation of powerful agent frameworks, a notable gap persists in the systematic analysis and standardization of their underlying architectures. While specific platforms like AIOS demonstrate how to create cohesive environments for autonomous agents [12], and frameworks such as MAToM-SNN leverage novel neural networks for complex agent interactions [13], they represent bespoke implementations rather than generalizable architectural knowledge that can guide future designs. Several recent surveys have highlighted this challenge from different perspectives. For instance, some reviews provide a broad overview of agent frameworks and their capabilities, confirming the lack of standardized architectures as a barrier to integration [14]. Other studies focus on specific aspects like prompting techniques [15] or the challenges in multi-agent systems [16], often concluding with a call for more robust and standardized frameworks [17, 18].

While existing studies correctly identify the need for standardization, our work addresses a more specific and fundamental gap: the lack of a comprehensive, evidence-based taxonomy from a **software architecture perspective**. To our knowledge, no prior work has systematically catalogued the fundamental *architectural design options* for FM-based agents and explicitly analyzed their resulting *non-functional trade-offs*. Our work aims to fill this gap by offering a detailed taxonomy and a corresponding decision model. The framework provides a systematic approach by mapping these architectural options to a comprehensive set of non-functional qualities, ranging from core engineering attributes like Performance, Reliability, Robustness, Resilience, Security, Maintainability, and Compatibility, to human-centric concerns such as Usability and Responsible & Trustworthy AI.

3 Methodology: A Systematic Process for Taxonomy Development

To construct a rigorous and evidence-based taxonomy for FM-based agent architectures, we adopted the systematic taxonomy development method proposed by Usman et al. [19]. This established process provides a transparent and

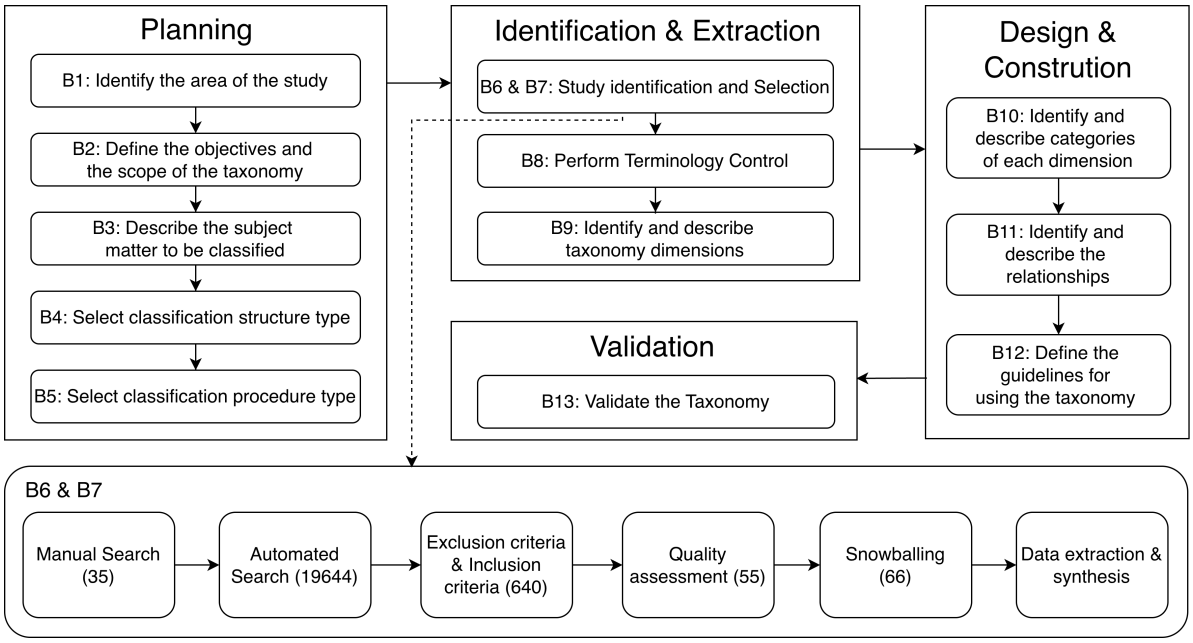


Figure 1: Methodology. The steps labeled B1-B13 are adopted from the taxonomy development process proposed by Usman et al. [19].

reproducible framework, ensuring that our taxonomy is grounded in a comprehensive analysis of the existing literature. The methodology consists of four main phases—Planning, Identification & Extraction, Design & Construction, and Validation—which are detailed in the following subsections. The initial evidence collection for this process was conducted through the SLR [20], which allowed us to systematically gather and analyze architectural characteristics from primary studies. This entire process is guided by our research questions:

- **RQ1: What are the fundamental architectural elements and design options that characterize existing FM-based agents?**

  This initial question drives the descriptive phase of our systematic literature review. It aims to identify and catalogue not just the capabilities of agents, but the core building blocks—the foundational components, patterns, and configurable choices—that constitute their *architecture*.

- **RQ2: How do the choices among these architectural elements create design trade-offs and influence system-level qualities?**

  This question moves from description to analysis, targeting the core of architectural reasoning. It investigates the consequences of design decisions, seeking to understand how selecting a specific architectural option (e.g., a centralized coordinator) impacts critical system-level qualities (e.g., performance, scalability, reliability) and creates inherent *architectural trade-offs*.

- **RQ3: How can a structured classification of these elements and trade-offs be utilized to guide the architectural design and decision-making for FM-based agents?**

  This final question addresses the practical utility of our work. It assesses whether a systematic taxonomy of the identified elements (from RQ1) and their associated trade-offs (from RQ2) can serve as an effective framework to not only analyze existing systems but also to *guide architects* in designing the next generation of FM-based agents, a question directly addressed by our proposed Decision Model.

3.1 Planning

The planning phase defines the taxonomy's context and initial setting. According to the guidelines from Usman et al. [19], this phase consists of five key activities that establish the foundation for the entire development process.

3.1.1 B1: Identify the Area of the Study

The knowledge area for this taxonomy is Software Engineering, with a specific focus on the software architecture of FM-based agents.

3.1.2 B2: Define the Objectives and the Scope of the Taxonomy

The primary objective of this study is to develop a comprehensive taxonomy that systematically classifies the architectural options for FM-based agents. The taxonomy is designed to address challenges in the design and implementation of these agents by integrating both functional and non-functional aspects. Its scope is therefore defined to include architectural decisions at both **design-time** (e.g., selecting a coordination model) and **run-time** (e.g., dynamic tool selection), covering the key stages of an agent's lifecycle.

3.1.3 B3: Describe the Subject Matter to be Classified

The subject matter of this taxonomy is the architecture options for FM-based agents. As defined in Section 1.2, an architecture option is a specific design approach adopted to address a particular architectural concern. Our subsequent SLR aims to identify, extract, and synthesize concrete instances of these architectural options from the existing literature, encompassing both the core operational mechanisms and the system-level quality attributes that characterize them.

3.1.4 B4: Select Classification Structure Type

This study adopts a **faceted taxonomy** approach [21, 22]. This structure supports classification across multiple independent dimensions and is well-suited to the evolving nature of FM-based agents, as it provides flexibility in representing their diverse attributes. The two primary facets of our taxonomy—**Functional Capabilities** and **Non-functional Qualities**—directly correspond to the dimensions of the subject matter identified in B3.

3.1.5 B5: Select Classification Procedure Type

This study employs a qualitative classification procedure, utilizing **thematic analysis** as the core technique to synthesize the data extracted from our SLR [23, 24]. Specifically, in the Design & Construction phase (Section 3.3), we apply an iterative process of coding and grouping to the extracted architectural features. This process allows us to systematically identify recurring themes, which then form the basis for defining the dimensions (Step B9) and categories (Step B10) of our taxonomy. To mitigate individual bias, we adopted a negotiated agreement approach for the qualitative classification. The initial classification was performed by one researcher and then thoroughly reviewed by a second researcher. Any disagreements were subsequently resolved through discussion and consensus among all authors.

3.2 Identification & Extraction

3.2.1 B6 & B7: Study Identification and Selection

To ensure a comprehensive and replicable selection of primary studies, we adopted the Quasi-Gold Standard (QGS) search strategy approach [20]. This is a well-established methodology in software engineering systematic reviews that aims to maximize coverage and relevance. As illustrated in the exemplar study by Hou et al. [25], the QGS process involves three main steps:

- A manual search of top-tier venues to build a "quasi-gold standard" set of highly relevant papers.
- Derivation of a comprehensive search string based on the keywords and terminology used in the QGS set.
- Execution of a broad automated search across multiple digital libraries using the derived search string.

Manual Search: We began by performing a manual search in top-tier software engineering venues, selected for their high impact and relevance to software architecture and engineering research. The selected venues are: ASE (IEEE/ACM International Conference on Automated Software Engineering), FSE (ACM SIGSOFT Symposium on the Foundations of Software Engineering), TOSEM (ACM Transactions on Software Engineering and Methodology), ACM Computing Surveys, ICSE (IEEE/ACM International Conference on Software Engineering), ISSTA (ACM SIGSOFT International Symposium on Software Testing and Analysis), TSE (IEEE Transactions on Software Engineering), and JSS (Journal of Systems and Software). From these venues, we systematically identified an initial set of 35 primary studies that directly addressed the architecture of FM-based agents. This collection of 35 papers served as our Quasi-Gold Standard (QGS) set, forming an evidence-based foundation for developing our search string. Using

the 35 papers in the QGS set as a ground truth, we performed a systematic keyword analysis with the assistance of a Large Language Model (LLM) to systematically extract and categorize prevalent terms and their synonyms (e.g., specific SE tasks like automated program repair) from the QGS texts. The detailed few-shot extraction prompt and the complete data logs are publicly available in our replication package. The final search string is composed of two main conceptual pillars, designed to be combined with a Boolean AND operator. Pillar A covers the core technology (the agent), while Pillar B covers the domain and discipline (the architecture and SE tasks). The keywords within each pillar are combined with a Boolean OR operator.

- **Pillar A: The Technology (FM-based Agent)**
  Keywords included: *Foundation Models, Large Language Models (LLMs), Generative AI, Conversational AI, Transformer Models, Autonomous Agents, AI, Multi-agent Systems (MAS), LLM-based Agents, Generative Agents, Autonomous Web Agent (AWA).*
- **Pillar B: The Domain/Discipline (AI Agent Architecture & SE Task)**
  Keywords included: *Agent-based Systems, Modular Architectures for AI Agent, AI Agent Design, Autonomous Agent System Design, Adaptive AI Architectures, Adaptive AI Taxonomy, Software Architecture, System Design, Code Generation, Automated Program Repair (APR), Bug-Fixing, Fault Localization, Software Testing, Code Review, Requirements Engineering, Software Maintenance, Refactoring, Code Snippet Adaptation, Code Intent Extraction, Simulation Testing, Security, Prompt Engineering.*

These pillars report the candidate terms produced by the extraction step, rather than a single search string applied uniformly across all databases. As some extracted SE-task terms were broader than the architectural focus of our research questions, not all candidate terms were included in the final queries. We selected the terms most relevant to FM-based agents and their architectures when constructing the executable queries. The query format was then adapted to the supported fields and operators of each digital library. The exact queries executed for each database are available in our replication package.

Automated Search: With the derived search string, we conducted the automated search across seven major digital libraries to ensure broad coverage of the relevant literature. The search was configured to include papers published between July 2017 and July 2025, covering the period since the emergence of modern transformer-based architectures[26]. This process retrieved 822 articles from the ACM Digital Library, 2,990 from IEEE Xplore, 909 from ScienceDirect, 4,456 from Web of Science, 5,019 from SpringerLink, 4,670 from arXiv, and 778 from DBLP. Using the above search strategy, we initially obtained 19644 articles that potentially relate to our research.

Exclusion & Inclusion Criteria: Next, we further evaluated the relevance of these articles based on the inclusion and exclusion criteria shown in Table 1 [27, 28]. The article selection process, as illustrated in Figure 1, consists of several phases. In the first phase, we conducted an automated filtering to exclude articles with fewer than eight pages (Exclusion Criterion 1), reducing the number of articles to 13991. In the second phase, we examined the titles and abstracts of the articles to identify those that directly relate to our research topic. The purpose of this phase was to apply our core inclusion criteria, ensuring that the studies discuss both FM-based agents and their application or architecture in software engineering (Inclusion Criteria 1, 2). We also excluded non-English literature during this phase (Exclusion Criterion 6). After the second phase, the number of articles was reduced to 4841.

The third phase involved identifying the venues and types of articles. We excluded studies belonging to books, theses, editorials, and other non-primary sources (Exclusion Criterion 3), as well as those published in workshops or doctoral symposiums (Exclusion Criterion 4). We also applied a filter for grey literature, such as non-refereed technical reports (Exclusion Criterion 5). We extracted publication information such as "journal," "URL," "DOI," and "series" to determine the publication sources. For articles from arXiv from 2023 to 2025, we decided to keep them, since this field is emerging and many works are in the process of submission. To mitigate the risks of including non-peer-reviewed work from 2023-2025, we implemented a stringent quality assessment (QA) protocol (Table 2). This step resulted in 1851 articles. In the fourth phase, we merged and deduplicated the remaining articles from the seven databases (Exclusion Criterion 2), resulting in 1588 unique articles. We then reviewed the full texts of these articles. This phase primarily involved excluding articles that mentioned FM-based agents only peripherally or without describing the employed techniques and architecture (Exclusion Criteria 7, 8). By further assessing the quality of the articles, we identified 640 articles directly relevant to our research. GPT-4o was used only as an assistive tool during screening. At each screening round, the primary researcher reviewed the model-generated filtering results before proceeding, and all final inclusion and exclusion decisions were made by the researcher. To ensure transparency, all prompts, and raw filtering records are provided in our replication package for verification. To ensure reliability, a second senior researcher audited the inclusion/exclusion decisions for the candidate set of 640 papers. This process achieved an

Table 1: Inclusion and Exclusion Criteria for Study Selection

Inclusion criteria	
(1)	The article claims that a Foundation Model or Large Language Model based agent is used.
(2)	The article claims that the study involves a Software Engineering (SE) task or discusses software architecture.
Exclusion criteria	
(1)	Short articles whose number of pages is less than eight.
(2)	Duplicate articles or similar studies with different versions from the same authors.
(3)	The article is not a primary research paper (e.g., books, theses, keynotes, editorials, panels).
(4)	The article is published in a workshop or a doctoral symposium.
(5)	The article is a grey publication (e.g., a non-peer-reviewed pre-print) published before 2023.
(6)	The article is not written in English.
(7)	The article mentions the use of FM-based agents without describing the employed techniques or architecture.
(8)	Studies where the primary contribution is the empirical evaluation of an agent's performance on an SE task, rather than a novel contribution to the agent's architectural design, patterns, or principles.

agreement rate of 92% (calculated specifically on the candidate set, excluding papers previously filtered by GPT-4o), with any disagreements resolved through discussion.

Quality Assessment: To ensure the reliability and validity of the selected primary studies and to mitigate potential biases introduced by low-quality research, we conducted a formal quality assessment (QA) [25]. This process is a standard practice in systematic literature reviews. We designed a checklist of 10 Quality Assessment Questions (QAQs), shown in Table 2, inspired by established SLR guidelines and tailored to the specific goals of our research on agent architectures. The assessment was conducted in a two-stage process. In the first stage, QAQ1 and QAQ2 served as a final check on our inclusion criteria. Each question was answered with a simple **Yes (1 point)** or **No (0 points)**. A study had to receive a score of 1 or 2 to proceed to the second stage of quality scoring; if the score was 0, it was excluded. In the second stage, we evaluated the quality and rigor of the remaining studies using QAQ3 through QAQ10. Each of eight questions was scored on a three-point scale: **Fully Met (2 points)**, **Partially Met (1 point)**, or **Not Met (0 points)**. To account for the inclusion of recent pre-prints from arXiv, which is a common practice in this fast-evolving field, we applied a differentiated scoring threshold:

- **For peer-reviewed articles:** All eight questions (QAQ3–QAQ10) were scored, leading to a maximum possible score of 16. We included studies that achieved a total score of **12 or higher** (approximately 75%).
- **For non-peer-reviewed pre-prints (e.g., from arXiv):** QAQ9 was treated as not applicable because it assesses peer-review status and venue standing, which cannot yet be observed for a pre-print. The score was therefore calculated using the remaining seven questions, resulting in a maximum score of 14. Pre-prints were included if they achieved a score of at least 10 out of 14 (approximately 71.4%).

For QAQ9, venue standing was assessed using recognised indicators relevant to the venue type, including CORE rankings for conferences, JCR or SJR rankings and impact indicators for journals, and the recognised standing of the venue within the software engineering and AI research communities. A score of 2 was assigned to an established peer-reviewed venue, a score of 1 to a peer-reviewed venue with limited or emerging evidence of established standing, and a score of 0 when the venue's peer-review status or standing could not be verified.

QAQ9 assesses peer-review status and venue standing rather than the methodological quality of the study itself. For pre-prints, this dimension was treated as not applicable because the peer-review and publication process had not been

Table 2: Checklist of Quality Assessment Questions (QAQs)

ID	Quality Assessment Question
— Stage 1: Final Relevance & Scope Check —	
QAQ1	Does the study definitively propose, implement, or analyze an agent based on Foundation Models (FMs) or Large Language Models (LLMs)?
QAQ2	Is the study a primary research paper (i.e., not a survey, systematic review, tutorial, or editorial)?
— Stage 2: In-depth Quality & Rigor Assessment —	
QAQ3	Is the motivation for the research and the problem statement clearly defined?
QAQ4	Does the study provide a clear and understandable description of the agent's architecture (e.g., its components, connectors, and their relationships)?
QAQ5	Is the research methodology (e.g., case study, experiment, empirical study) clearly described and appropriate for the research questions?
QAQ6	Does the study include an evaluation or a detailed discussion of the agent's capabilities or the trade-offs of its architectural choices?
QAQ7	Is there a clear statement of findings?
QAQ8	Is the research novel and does it contribute to the field?
QAQ9	Was the paper published in a reputable, peer-reviewed venue (e.g., a well-known conference or journal)?
QAQ10	Are the limitations of the study or threats to validity clearly described?

completed at the time of data collection. Following Hou et al. [25], their quality scores were therefore calculated using the remaining seven questions. This treatment does not imply that a pre-print is equivalent to a paper published in a reputable venue; it only indicates that the venue-related dimension could not yet be assessed.

This structured and transparent scoring rubric ensures that all included studies, regardless of their publication status, meet a consistent and high standard of quality and relevance to our research. Specifically, 47 papers were excluded at Stage 1 (QAQ1-QAQ2) and 538 at Stage 2 (scoring). The scoring was performed by a primary researcher and audited by a senior researcher, with any discrepancies resolved through negotiated agreement. The filtering records for both stages and the detailed scoring for the final studies are provided in our replication package. After this quality assessment, a final set of 55 primary studies were selected for data extraction.

Snowballing: To ensure the comprehensiveness of our study and to identify any relevant primary studies that may have been missed during the database search, we conducted a snowballing search process. Snowballing is a systematic technique that involves using the reference list of an article (backward snowballing) and the citations to an article (forward snowballing) to identify additional relevant literature [19]. The starting point for this process was the final set of 55 primary studies that passed our quality assessment. We performed both backward and forward snowballing on this entire set. The backward pass, which involved scanning the reference lists of our included studies. The forward pass involved identifying all articles that cited our included studies (using Google Scholar's "Cited by" feature). After merging and deduplicating these two sets, this new set of articles was then subjected to the exact same multi-phase screening and quality assessment process described in the above steps. This rigorous re-evaluation resulted in the inclusion of an additional 11 primary studies. By combining the results from our automated database search and the snowballing search, the final set of primary studies selected for data extraction and synthesis consists of **66** articles (**52** peer-reviewed articles and **14** pre-prints).

3.2.2 B8: Perform Terminology Control

During the data extraction phase (B7), it became evident that the primary studies used a diverse and often inconsistent set of terms to describe similar architectural concepts, a common challenge in a rapidly evolving field like FM-based agents. To ensure the conceptual integrity and prevent ambiguity in our taxonomy, a systematic Terminology Control

step (B8) was essential [19]. The goal of this step was to systematically harmonize synonymous or overlapping terms into a single, consistent concept that would be used throughout our taxonomy.

Our process involved using the data extraction form to explicitly document the original terminology used in each paper alongside the architectural concept it described. Through an iterative process of thematic analysis and consensus among the research team, these source terms were mapped to a standardized set of categories. A prime example of this process can be seen in the dimension of *Agent Coordination*. Our review of the literature revealed multiple terms describing architectures where a single agent manages or directs others. For instance, we encountered phrases such as *'orchestrator model'*, *'hierarchical control'*, *'master-slave pattern'*, and *'central controller'*. These were all systematically unified under the single, standardized category of **Centralized** coordination within our taxonomy. Similarly, for coordination patterns without a central authority, terms like *'peer-to-peer (P2P) interaction'*, *'decentralized consensus'*, and *'emergent collaboration'* were identified and harmonized into the category of **Fully Distributed** coordination.

This rigorous process of terminology control was applied to all concepts extracted from the literature, ensuring that the final taxonomy is built upon a clear, consistent, and well-defined conceptual foundation.

3.2.3 B9: Identify and Describe Taxonomy Dimensions

Following terminology control (B8), the next step was to identify the taxonomy's high-level dimensions from the synthesized data. This was not a pre-determined step but an emergent process driven by the evidence from the primary studies. We employed a thematic analysis approach, using an iterative process of open and axial coding on all extracted architectural features to group related concepts into higher-level themes. This bottom-up analysis led to the emergent definition of the **Functional Capabilities** dimension. For the **Non-functional Qualities** dimension, we adopted a hybrid approach. We began with a top-down synthesis of candidate qualities from established standards, including **ISO/IEC 25010** [29] and the foundational work *Software Architecture in Practice* [30] and *Engineering AI Systems* [31]. We utilized these definitions as an initial codebook, performing a systematic deductive mapping to validate and refine them against the architectural evidence extracted from our primary studies. This process resulted in the two primary, orthogonal dimensions of our taxonomy, which are formally defined as:

- Functional Capabilities: Consolidates all architectural options that define an agent's core operational mechanisms (e.g., how it perceives, reasons, and acts).
- Non-functional Qualities: Groups the system-level quality attributes (e.g., reliability, performance) that constrain or define the overall quality of the architecture.

This data-driven derivation confirms the initial scope defined in B3 and provides an empirically grounded structure for the subsequent classification of categories in step B10.

3.3 Design & Construction

Following the establishment of the taxonomy's foundational dimensions, the subsequent steps focused on the detailed construction and rigorous validation of the classification scheme, adhering to the procedures B10 through B13 of the adopted methodology [19].

- **B10 & B11: Identify Categories and Their Relationships.** For each high-level dimension, specific categories representing distinct architectural options were identified through a systematic synthesis of the extracted data. Our design aimed for orthogonality at multiple levels of the taxonomy. Not only are the two primary dimensions (Functional Capabilities and Non-functional Qualities) conceptually independent, but the sub-dimensions within each category (e.g., Memory Management, Planning, and Action) are also designed to be mutually exclusive and non-overlapping. We acknowledged that strict mutual exclusivity is challenging in this domain; therefore, we resolved borderline cases by classifying components based on their primary architectural intent within the specific system context. This structural integrity was formally verified through our expert-driven validation process, as detailed in Section 6. The complete taxonomy is presented in Section 4.
- **B12: Define Guidelines for Using the Taxonomy.** To translate the descriptive taxonomy into prescriptive guidance for architects, we developed a decision model. This model is presented as a Decision Matrix in Section 5 and explicitly maps architectural choices to their impact on system qualities, thereby guiding practitioners through critical design trade-offs.

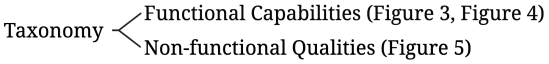


Figure 2: The high-level structure of the taxonomy. Two primary dimensions: Functional Capabilities and Non-functional Qualities.

3.4 Validation

While the rigorous SLR process ensured the traceability and groundedness of the taxonomy's construction, we relied on a formal, external validation as our primary validation mechanism to assess the taxonomy's completeness, correctness, and practical relevance. As detailed in Section 6, this external validation involved an expert-driven assessment using a structured questionnaire with participants from both industry and academia.

The replication package for this study, including the detailed search terms, the full list of screened studies with decisions, raw extracted data, expert validation protocol and results, and analysis scripts, is publicly available at[12].

4 Taxonomy Reporting and Validation

In this section, we present a taxonomy that defines architecture design options for FM-based agents. The taxonomy is structured into two categories: functional capability (Section 4.1 - Section 4.11) and non-functional qualities (Section 4.12). Figure 2 illustrates the high-level structure of this taxonomy, while detailed breakdowns of the functional capabilities are presented across Figure 3 and Figure 4, and the non-functional qualities are shown in Figure 5. We discuss the characteristics of each design option to consider during the building process and their impact on the FM-based agents. Additionally, we provide a mapping table 5 that maps each of our 66 primary studies to the corresponding taxonomy categories.

4.1 Input Modality

Modality defines whether an agent operates using a single type of data input or multiple. **Single-modality** agents process one type of input, such as text, vision, or audio [32]. This approach is well-suited for straightforward tasks where computational efficiency and simpler data interpretation are priorities, as seen in text-based chatbots or vision-only surveillance systems [33]. In contrast, **multi-modality** agents are architected to integrate and process a combination of inputs—including text, audio, images, and video—to gain a more holistic understanding of their operational environment [34, 35]. For instance, by processing simultaneous speech and visual gestures, these agents can provide more accurate and context-aware responses [36]. The capability to synthesize diverse data streams is a critical architectural driver for complex applications, such as autonomous navigation or advanced interactive assistants, which demand high levels of adaptability and contextual awareness.

4.2 Context

Context management focuses on gathering and organizing information about the agent's operating environment to understand user intentions and goals better. Our taxonomy classifies this into two sub-dimensions: the types of context gathered and the strategies for goal seeking. The classification of **context types** is based on the nature of the information itself. `Interactional context` refers to direct user actions and behaviors, such as mouse clicks, typing patterns, and gestures [37]. `Environmental context` describes the state of the digital environment in which the user is operating [38]. Finally, `Semantic context`, extracted from sources like user annotations or prompt content, pertains to the underlying meaning and intent, providing crucial insights for goal interpretation [39]. For **goal seeking**, agents can employ two primary strategies. With `passive suggestion`, the agent analyzes goals that are explicitly articulated by the user, typically through a dialogue interface [35]. In contrast, `proactive suggestion` goes beyond explicit commands, enabling the agent to anticipate user goals by interpreting multimodal context from UI elements and user interactions [40]. The architectural choice between these strategies presents a fundamental trade-off: passive suggestion offers implementation simplicity, whereas proactive suggestion enhances the agent's autonomy and intelligence at the cost of greater system complexity.

[12]https://github.com/submissionpurposeonly/jss/tree/main

4.3 Role-Playing

In multi-agent systems, assigning distinct roles is a crucial architectural decision that defines the functions and interactions within the system. The taxonomy categorizes these roles into two primary types: **agent-as-a-coordinator** and **agent-as-a-worker**. An **agent-as-a-coordinator** primarily formulates high-level strategies and orchestrates task execution by delegating responsibilities to other agents or external systems, ensuring the entire system operates cohesively and efficiently [1, 5, 10, 41]. In contrast, an **agent-as-a-worker** acts as the core executor. Upon receiving assignments, it is responsible for generating local, sub-task level strategies to complete them [34, 42–46]. To achieve its objectives, a worker agent may operate with varying degrees of autonomy, interact with other agents through cooperation or competition, and call external tools. This architectural pattern enables a clear separation of concerns, allowing for strategic oversight by coordinators while leveraging the specialized, tactical execution of workers to solve complex problems.

4.4 Goal Type

Agents establish goals to guide their planning and action phases, which our taxonomy classifies into three primary types: task completion, communication, and learning. **Task completion** goals direct an agent to achieve specific, complex objectives. Examples include agents programmed to craft items in virtual environments like Minecraft [4, 47] or to execute predefined functions during software development [48]. **Communication** goals involve agents interacting with other agents or humans to exchange information or collaborate on joint tasks. For instance, agents within platforms like ChatDev coordinate their efforts on development tasks [48], while others are designed to adapt their strategies based on real-time human feedback. Finally, **learning** goals require agents to navigate and adapt to unfamiliar settings, balancing the exploration of new areas against the exploitation of known resources. The Voyager agent exemplifies this, as it explores and refines its skills through continual feedback and adjustment processes [4].

4.5 Planning

In FM-based agents, planning is operationalized through a reasoning process, where the agent uses cognitive steps and logical frameworks to solve complex problems [43, 49]. This process bridges perception and action by enabling informed decision-making, which is implemented through architectural choices in how the agent selects tools and generates plans.

4.5.1 Planning Engine

The planning engine is the central component orchestrating high-level strategic activities. It enables the agent to process reasoning, generate and monitor plans, and adapt to environmental shifts [50]. Architectural options include:

- **Internal Planner:** Embedded within the agent's core architecture, this planner uses the agent's intrinsic capabilities to autonomously generate and execute plans [51].
- **External Planner:** Also known as classical planners, these are specialized, often domain-specific tools integrated to handle complex tasks requiring detailed and precise planning logic [52, 53].
- **Hybrid Approach:** This approach combines an internal FM-based planner for flexible problem translation with a classical external planner for efficient and reliable execution, leveraging the strengths of both [53].

4.5.2 Tool Selection

During planning, an agent must select the appropriate tools to execute its plan. The architectural choice lies between using tools that are internal to the agent's design or external to it.

- **Internal Tools:** These are capabilities and algorithms built directly into the agent's system to provide tailored solutions. Examples include proprietary LLMs for reasoning, or specialized, self-contained algorithms and knowledge bases, such as those developed for the Voyager agent to manage its skills [4, 54].
- **External Tools:** These are outside services and data sources that the agent interacts with, typically via APIs. This allows the agent to extend its capabilities beyond its inherent functions, for instance, by calling a web search API, a code interpreter, or other external computational models to acquire new information or perform specialized tasks [54].

4.5.3 Plan Generation

Plan generation translates the reasoning process into a sequence of actionable steps for the agent to follow [55]. There are two primary architectural options for this process:

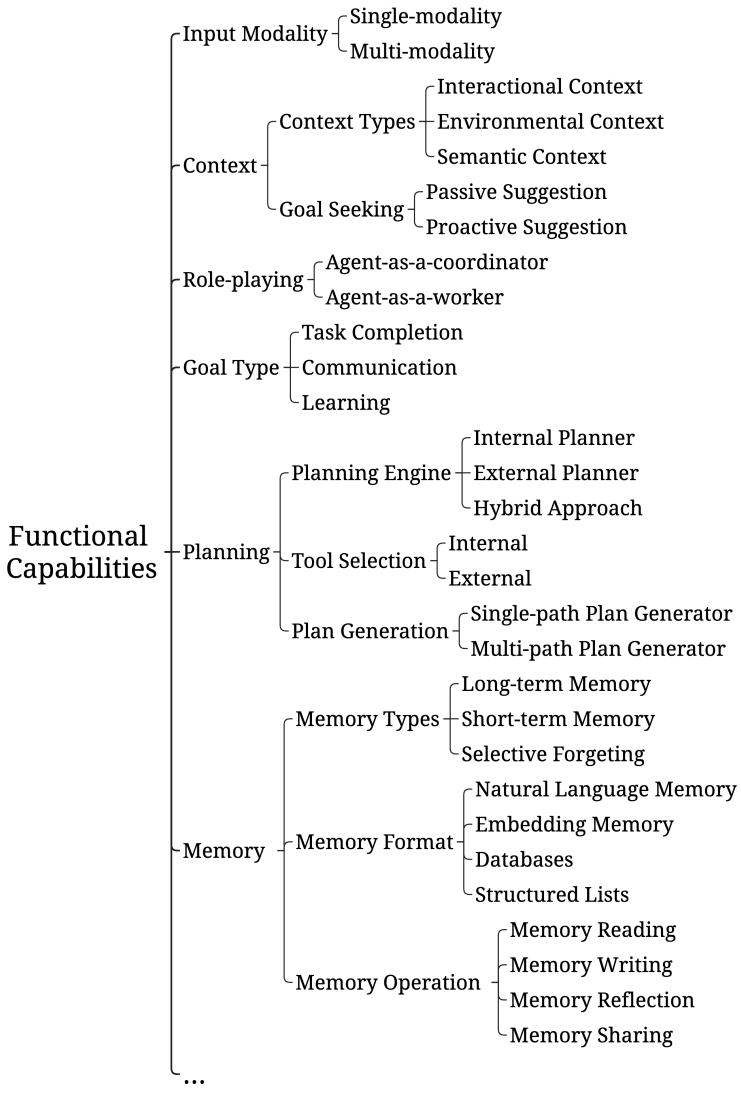


Figure 3: The taxonomy of Functional Capabilities (Part 1 of 2), detailing architectural options from Input Modality to Memory.

- **Single-path Plan Generator:** This approach creates a linear, step-by-step plan. It is ideal for tasks in predictable environments where a straightforward procedure is effective, ensuring an efficient and direct route to the goal [56, 57].
- **Multi-path Plan Generator:** In contrast, this approach devises several potential routes to achieve a goal. It allows the agent to dynamically adjust its plan based on new information, making it more flexible and robust in complex or uncertain scenarios [51, 58–60].

The choice between these generators represents a critical architectural trade-off between the efficiency and predictability of single-path plans and the adaptability and resilience of multi-path plans.

4.6 Memory

Memory is a critical architectural dimension that enables FM-based agents to store, retrieve, and utilize information from past interactions to inform current and future actions. An agent's ability to maintain context, learn from experience, and exhibit consistent behavior over time is fundamentally dependent on the design of its memory system. This

section details the architectural options related to memory, including its types, the formats used for storing information, and the core operations for manipulating it.

- **Memory Types: Long-term memory** retains information over long periods, which is essential for tasks that require historical data, knowledge, past observations, and past experiences [57, 61]. **Short-term memory**, on the other hand, handles short-term information relevant to immediate tasks [62]. This type of memory is useful for temporary activities that do not require long-term storage, for example, configuration, working context, recent events, and the information within the context window of the FM. **Selective forgetting** enables agents to discard irrelevant or outdated information, ensuring that they maintain optimal performance without being bogged down by unnecessary data [62]. This capability is crucial in dynamic environments where the relevance of data changes rapidly.
- **Memory Format:** In the FM-based agent systems, several distinctive structures offer unique advantages and serve as pivotal design options depending on specific application needs. **Natural language memory**, as used in systems like Reflexion [63] and Voyager [4], provides a flexible format that stores information in an easily understandable form, facilitating intuitive interaction and preserving rich semantic details to guide agent actions. **Embedding memory**, exemplified by MemoryBank and ChatDev, encapsulates memory information into compact embedding vectors, significantly enhancing the efficiency of memory retrieval processes [64]. **Databases** allow for robust memory manipulation; systems like ChatDB [48] and DB-GPT use databases to enable precise memory operations through SQL queries, providing structured and efficient data management. **Structured lists** are employed in models such as GITM and RET-LLM, where memory is systematically organized in lists or hierarchical structures that clearly define the relationships between elements and facilitate rapid data access [39, 62]. Each format presents distinct advantages: natural language for clarity and semantic richness, embeddings for retrieval speed, databases for structured manipulation, and lists for organized and hierarchical memory storage. These memory formats can be integrated, as demonstrated by GITM, which uses a hybrid approach combining key-value pairs with embeddings and natural language to maximize retrieval efficiency and content comprehensiveness, providing multiple design options to optimize agent performance based on the specific needs of the application.
- **Memory Operation:** Agents can acquire, accumulate, and utilize knowledge through interactions with their environment via various operations. These operations include memory reading, writing, reflection, and sharing. **Memory reading** involves extracting valuable information based on recency, relevance, and importance to enhance the agent's actions [42, 43, 46, 62]. **Memory writing** focuses on storing perceived environmental information while managing duplication and preventing overflow [65]. **Memory reflection** enables agents to summarize and infer high-level insights from past experiences, facilitating more abstract and complex decision-making [62]. **Memory sharing** allows agents to access and contribute to a common memory pool. This enhances their collaborative abilities by integrating diverse experiences and knowledge [12, 61, 66, 67]. In FM-based agent systems, memory sharing involves storing and retrieving shared memories, which supports continuous learning and adaptation, helping agents use the most relevant information for tasks.

4.7 Action

The Action dimension encompasses the architectural choices that govern how an agent executes tasks and interacts with its environment, other agents, or external tools. It represents the phase where the agent translates its internal plans and decisions into tangible operations that effect change. This section describes the key architectural options within this dimension, including the mechanisms for agent coordination, the strategies for communication, the methods for tool calling, and the types of actuation.

4.7.1 Agent Coordination

At runtime, agent coordination mechanisms ensure that multiple agents can work together effectively, avoiding conflicts and redundancies. The taxonomy identifies three primary coordination patterns:

- **Centralized Coordination:** This pattern follows a strict command-and-control model where a single "manager" agent dictates specific tasks and steps to other agents, which have limited autonomy. The system's core intelligence is concentrated at the center [12, 16].
- **Federated Coordination:** In this model, a central "orchestrator" is responsible for high-level tasks like assigning goals or aggregating results, but it grants high autonomy to individual agents to manage their own internal execution logic and complex sub-tasks [5, 42, 43, 46, 68].

- **Fully Distributed Coordination:** This approach involves no central authority; all agents are peers that coordinate directly through peer-to-peer communication to reach a consensus or synchronize actions [16, 47, 62].

The selection among these patterns represents a fundamental architectural trade-off between the simplicity and efficiency of centralized control versus the resilience and autonomy inherent in distributed systems.

4.7.2 Agent Communication

Agent communication strategies dictate how agents share information to collaborate, representing a crucial aspect of multi-agent system design. The primary architectural choice concerns the level of transparency:

- **Full Transparency:** All information is openly shared among agents to maximize cooperation and situational awareness [57, 58, 67]. While ideal for deeply collaborative tasks, this approach can introduce risks of information overload, privacy breaches, and security vulnerabilities [69].
- **Partial Transparency:** Information sharing is restricted to enhance security and efficiency. This is achieved through several patterns, including: **Goal-based sharing**, where information is exchanged only if relevant to specific goals [16]; **Role-based sharing**, where access is determined by an agent's responsibilities [16]; **Sensitive data withholding** to protect privacy [69, 70]; and **Context-aware sharing**, where the decision to share is adapted dynamically based on the system's state [70–72].

Architecting the communication strategy therefore requires balancing the need for information availability against the critical system qualities of security, privacy, and efficiency.

4.7.3 Tool Calling

Tool calling is a fundamental capability that enables agents to access and manipulate external tools. This dimension is composed of several key architectural choices.

Tool Invocation defines how an agent interacts with a tool. With `Configuration-free` invocation, agents use tools via predefined interfaces without adjustment, ideal for standardized, routine operations [73]. In contrast, `Customizable invocation` allows agents to dynamically adjust tool parameters to optimize performance for specific tasks [73–75]. For example, a code generation agent can customize a `create_function` tool by specifying parameters like `language='Python'` and `args=['user_id']`, enabling both high precision for current requirements and adaptability for future changes.

Tool Interface is the layer through which an agent operates a tool. An `API` provides a structured, machine-to-machine interface, enabling reliable and efficient interaction with services that expose a programmatic endpoint [17, 54, 73, 76–78]. Alternatively, `UI Understanding` allows an agent to operate a tool by perceiving and interacting with its user interface, mimicking human behavior [74]. This is crucial when no API is available [79] and can be applied to multimodal interfaces, including visual, audio, or gesture-based systems [79, 80].

Tool User Type determines the source of feedback for tool refinement. `User experience-driven` tools evolve based on direct human interaction and feedback, prioritizing usability and user satisfaction [81, 82]. `Agent experience-driven` tools, on the other hand, harness the agent's own historical performance data to autonomously refine and optimize operational strategies [50]. This choice represents an architectural trade-off between optimizing for human-centric usability and for agent-centric autonomous performance.

Tool Learning describes how agents acquire new tool-use capabilities. In `API-based learning`, agents learn to master stable, structured interfaces, leading to highly reliable and efficient tool use [54, 73, 77]. In `UI-based learning`, agents learn to interpret and adapt to potentially inconsistent or evolving user interfaces, which grants greater flexibility at the cost of potential brittleness [35, 74].

Output Integration is the process of incorporating tool outputs into an agent's workflow. `Inline integration` involves directly using a tool's output for immediate task execution [54, 73]. `Multi-source integration` is a more complex approach where outputs from multiple tools are synthesized, often with the agent's internal data, to produce more nuanced and comprehensive results [76].

Actuation refers to an agent's capability to execute actions that affect a physical or digital environment [83]. The taxonomy distinguishes between two types:

- **Physical actuation** involves actions in the real world, such as the robotic movements and manipulations required for factory assembly tasks or the coordinated steering, braking, and acceleration in an autonomous vehicle [38, 84, 85].
- **Virtual actuation** encompasses actions within software environments, including automated data entry, network configuration changes, or interacting with software applications [74, 78, 86, 87].

4.8 Reflection

Reflection is the architectural dimension that equips an agent with meta-cognitive capabilities to introspectively analyze its own performance and behavior in order to learn and improve. This process of self-assessment is crucial for enabling adaptive and robust agents that can correct their mistakes and refine their strategies over time. This section details the architectural options for reflection, including the artifacts that are reflected upon, the providers of feedback, and the specific mechanisms used for reasoning.

4.8.1 Reflected Artifacts

Reflection can target different artifacts generated during an agent's operation to ensure traceability and enable feedback at various stages. The primary architectural options include:

- **Workflow/Plan Generation:** This includes the agent's initial goals, prompts, and planning steps. Reflection at this stage assesses the validity and comprehensiveness of the agent's initial approach before execution [88].
- **Intermediate Result:** This artifact comprises the logs of reasoning processes, planning steps, and tool use during operation, allowing for reflection on the correctness and rationality of the agent's internal decision-making process [44].
- **Final Output:** This refers to the final result produced by the agent, which is evaluated against the initial goals to verify that the outcome meets the desired criteria [44].

4.8.2 Provider

Reflection is driven by feedback, which can originate from several providers [62]. The architectural choice of provider determines the source of guidance for plan refinement:

- **Self-reflection:** The agent generates its own feedback to evaluate and refine its plans, forming an internal loop of self-improvement [4, 34, 41, 43, 44, 63, 89].
- **Cross-reflection:** One or more external agents or FMs are used to provide feedback, leveraging diverse perspectives to critique and enhance the plan [58, 63].
- **Human Reflection:** The agent collects feedback directly from human users, which is crucial for aligning its behavior with human preferences and expectations [52, 68, 85].
- **Environmental Reflection:** The agent obtains feedback from the external world, such as task completion signals or observations after an action [46, 49, 78, 90]. For example, the ReAct framework uses thought-act-observation triplets for planning [41], while Voyager incorporates feedback from program execution outcomes and errors [4].

4.8.3 Mechanisms

Mechanisms are the specific techniques an agent employs to structure its reflection process. The primary architectural options are:

- **Chain of Thought (CoT):** This mechanism involves the agent articulating its reasoning process step-by-step, which helps in tracing decisions and identifying errors [59, 77]. Advanced variants like CoCoNut extend this reasoning into a continuous latent space for greater efficiency [91].
- **Tree of Thoughts (ToT):** This more advanced mechanism allows an agent to explore multiple reasoning pathways simultaneously. By considering various possible outcomes before making a final decision, ToT enhances the robustness and depth of the agent's problem-solving capabilities [51, 60].

4.9 Learning Capability

Learning capabilities are fundamental to an agent's ability to adapt and improve its performance over time. The primary architectural options for enabling learning are:

- **Self-learning:** This capability allows an agent to autonomously improve by updating its knowledge base and refining its decision-making algorithms. It often relies on long-term memory to learn from extensive historical data, enabling performance enhancement without direct external input [61, 63, 92, 93].
- **In-context learning:** This enables an agent to adapt to new tasks on-the-fly by interpreting the provided context, typically through prompt engineering, without altering its internal model parameters [51, 52, 57, 62, 67]. This streamlined approach is highly effective for pre-trained models, allowing for rapid adaptation based on tailored prompts [94].
- **Group learning:** This involves multiple agents sharing knowledge and insights to enhance collective performance [58, 94]. It can be implemented through various architectural patterns, such as **peer-to-peer learning**, where agents share knowledge directly with each other [67, 72], or **hierarchical learning**, where information is processed and shared across different levels of an agent organization to solve complex problems more efficiently [58].

4.10 Workflow

The Workflow dimension governs the architectural pattern for the procedural execution of tasks. It is distinct from the strategic nature of *Planning* (Section 4.5) and the interactive focus of *Agent Coordination* (Section 4.7.1), as its concern is task and control flow. The primary architectural designs are:

- **Centralized Workflow:** Task management is concentrated at a single control point. This design enhances operational consistency but risks creating performance bottlenecks, a pattern analogous to centralized coordination [12].
- **Decentralized Workflow:** Task control is distributed across multiple agents. This approach increases system flexibility and resilience, reflecting principles found in distributed coordination strategies [47].

4.11 Access to Underlying Models

This dimension addresses the architectural choices related to the core engine of the agent: the foundation model(s) that provide its fundamental reasoning and generative capabilities. Decisions made in this dimension have a profound impact on the agent's core competence, cost, and operational constraints. This section details the key architectural options, including the types of underlying models, their composition, their sourcing, and the methods used for their training.

4.11.1 Underlying Model Types

The choice of the underlying model type determines the agent's fundamental scope of competence. The primary options are:

- **Domain-specific AI-based agents:** These agents are engineered to excel at a narrow set of specialized tasks within a particular domain, prioritizing high performance and accuracy over general applicability [95].
- **General Purpose AI (GPAI)-based agents:** In contrast, these agents are designed for versatility, capable of performing various tasks across different domains [32]. They leverage broad training data and continuous learning mechanisms to adapt to new and unforeseen challenges, prioritizing flexibility over specialized expertise [93].

4.11.2 Model Composition

Model composition defines how many and what kinds of models constitute the agent's core engine. The architectural choice is between a single model or a multi-model configuration.

- **Single-model:** The agent relies on a single, monolithic model to handle all of its tasks. This approach minimizes architectural complexity but may lack the scalability and specialized performance required for diverse tasks [16].

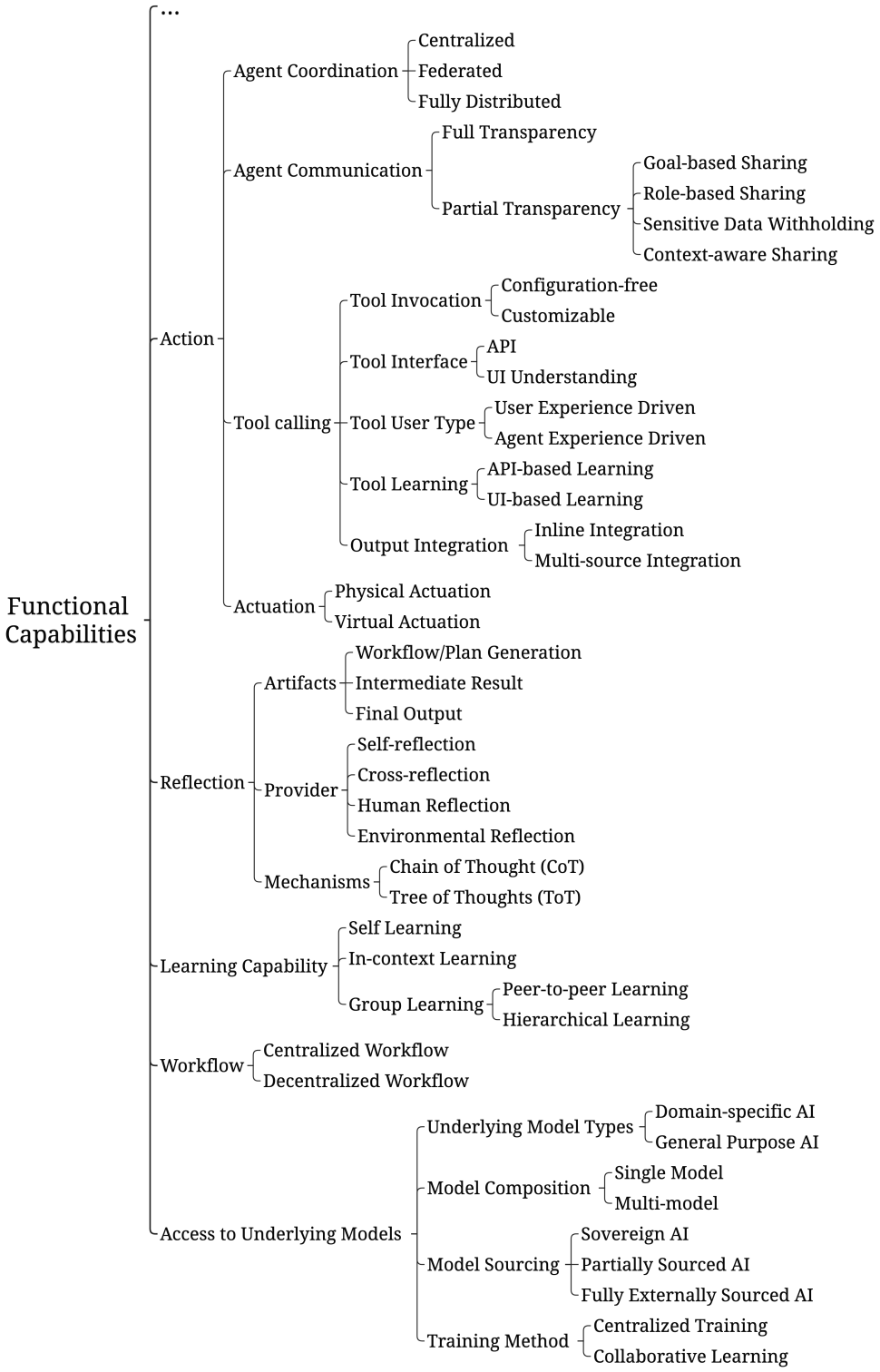


Figure 4: The taxonomy of Functional Capabilities (Part 2 of 2), detailing architectural options from Action to Access to Underlying Models.

- **Multi-model:** The agent employs multiple models to enhance performance, robustness, or task-specific capabilities [38, 42, 58]. This can be implemented through several advanced patterns, such as a **mixture of experts** for dynamic model selection [96], an **ensemble method** for improved accuracy [97], **model merging** to create composite capabilities [98], or a **hybrid model** that combines different model types (e.g., rule-based and learning-based) to leverage their respective strengths [99].

4.11.3 Model Sourcing

Model sourcing addresses the strategic decision of how an agent's core AI technology is acquired, offering different trade-offs between control, cost, and customization. The options are:

- **Sovereign AI:** The model is developed and maintained entirely in-house. This approach provides complete control over the model's capabilities and data security but requires significant investment [32].
- **Partially Sourced AI:** This hybrid approach combines internal development with external resources. For instance, BloombergGPT was developed by fine-tuning a base model on a unique mixture of proprietary financial data and public datasets to create a highly specialized, high-performance model [95].
- **Fully Externally Sourced AI:** The agent relies entirely on third-party AI models and services (e.g., via APIs), eliminating in-house development but ceding control over the model's architecture and evolution [77].

4.11.4 Training Method

The training method is a critical architectural decision based on the system's data architecture—that is, whether training data is pooled centrally or remains distributed. The primary paradigms are:

- **Centralized Training:** The traditional approach where a complete dataset is aggregated in a single location to train or fine-tune a model [32, 78, 86]. This ensures data consistency but can be constrained by resource requirements and privacy regulations.
- **Collaborative Learning:** This paradigm is used when data cannot be centralized, often due to privacy concerns [67]. It enables multiple parties to jointly train a model without sharing raw data, using patterns like **federated learning**, which shares model updates [100], which shares intermediate outputs from model layers.

While not parallel architectural choices in our taxonomy, two critical, cross-cutting concepts influence these paradigms and are essential for a complete architectural picture:

- **Distributed learning** is a **computational strategy** focused on performance; it uses multiple machines to accelerate any large-scale training and can be applied to both centralized and collaborative approaches.
- **Online learning** refers to the **update frequency**, allowing an agent's model to be continuously refined in real-time as new data arrives, a choice orthogonal to whether the data architecture is centralized or collaborative.

4.12 Non-functional Qualities

While functional capabilities define *what* an agent does, non-functional qualities determine *how well* it performs its functions. These qualities are critical system-level attributes that constrain the architectural design and dictate the overall quality and dependability of the agent. To provide a systematic and comprehensive classification, our taxonomy synthesizes principles from established software and AI systems architecture literature. It is primarily grounded in the **ISO/IEC 25010 standard** [29] for software product quality, enriched with foundational quality attributes described by Bass et al. in *Software Architecture in Practice* [30] and *Engineering AI Systems* [31]. To ensure transparency and validity, we employed a deductive mapping procedure. We used the definitions from these standards as a predefined coding scheme. During data extraction, architectural concerns identified in the 66 primary studies were systematically mapped to these categories. The key non-functional qualities relevant to FM-based agents are as follows:

Performance In the context of AI systems, performance is a multifaceted quality encompassing **Accuracy** and **Efficiency**. Accuracy measures the correctness of an agent's outputs, while Efficiency concerns the resources consumed (e.g., latency, computational cost, scalability) relative to the results achieved [97, 98].

Reliability This refers to the degree to which an agent performs its specified functions correctly under normal conditions. High reliability implies stable operation (*Maturity*), consistent service readiness (*Availability*), and accurate results for in-distribution inputs [47, 62].

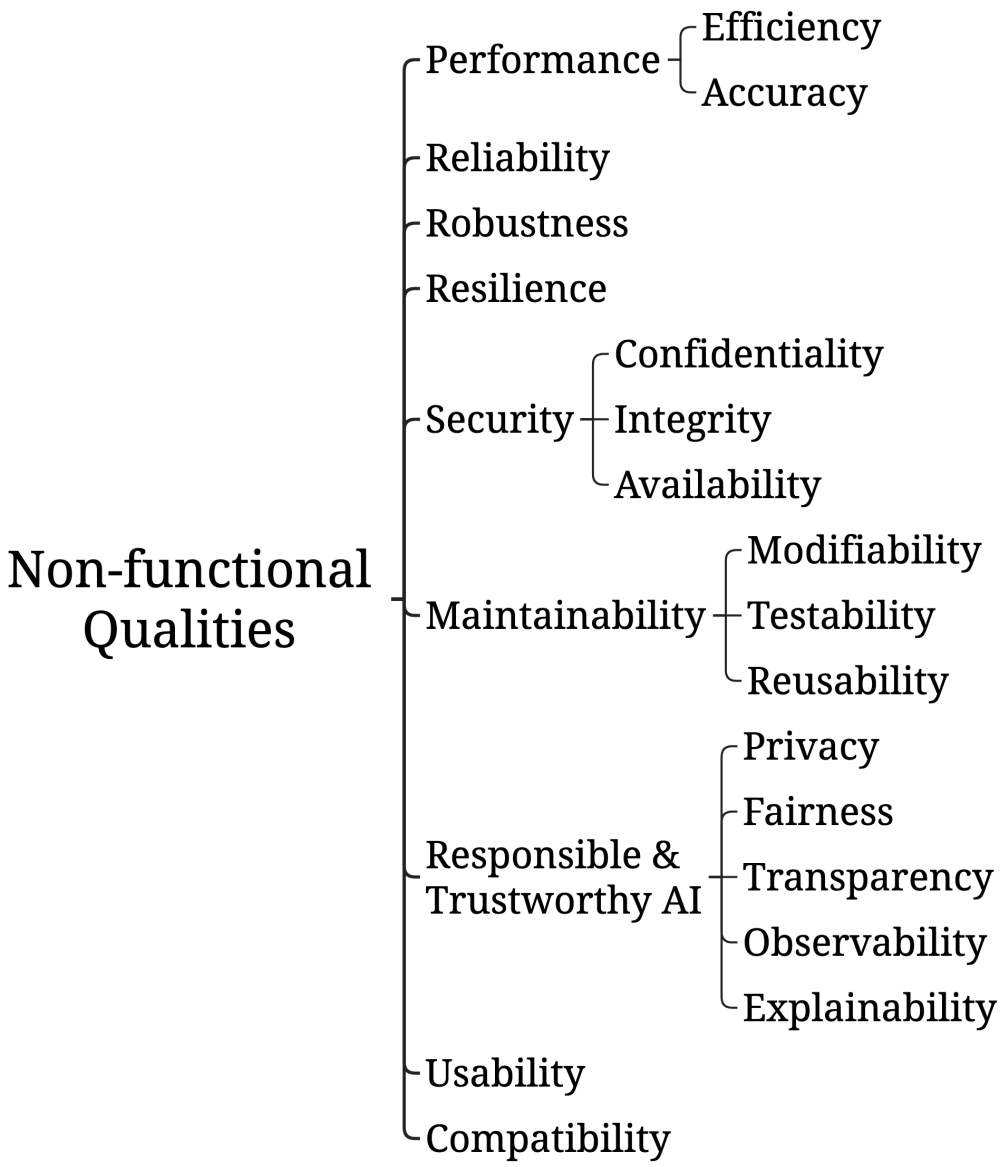


Figure 5: Taxonomy - Non-functional capabilities of foundation-model-based agent

Robustness Distinct from reliability, robustness is the agent's ability to maintain functionality in the presence of invalid, unexpected, or stressful inputs, such as noisy data or adversarial attacks—a critical quality for agents in unpredictable environments [51, 60].

Resilience Resilience is the ability of an agent to prepare for, absorb, recover from, and adapt to operational failures. It concerns the entire lifecycle of bouncing back from a significant failure, potentially to a degraded but still operational state [16].

Security This quality addresses the protection of the agent, its data, and services. It encompasses: **Confidentiality** (protecting data from unauthorized access), **Integrity** (safeguarding assets from tampering), and **Availability** (ensuring services are accessible on demand) [69, 70].

Maintainability This concerns the ease with which an agent's architecture can be modified to correct faults, improve performance, or adapt. It is characterized by **Modifiability** (ease of change), **Testability** (ease of verification), and **Reusability** (leveraging components in new contexts) [56, 57].

Responsible & Trustworthy AI (RAI) This crucial cross-cutting concern integrates ethics and accountability into the agent's design. It includes characteristics such as **Privacy**, **Fairness**, **Transparency**, **Observability**, and **Explainability** (the ability to provide human-understandable justifications) [1, 88].

Usability This quality addresses the effectiveness, efficiency, and satisfaction with which users can interact with an agent. It includes the intuitiveness of interaction, clarity of communication, and alignment with user intent [4, 63, 81].

Compatibility This quality refers to an agent's ability to exchange information and perform its functions while sharing an environment with other systems. It is vital for multi-agent systems and the integration of external tools via APIs [54, 76].

It is important to note that the **Level of Autonomy**, while a critical characteristic of an agent, is not classified here as a standalone non-functional quality. Instead, we consider it a crucial **emergent property** that arises from the specific combination of choices made within the **Functional Capabilities** dimension. An agent's ability to operate without human intervention is a direct consequence of its architectural design. Therefore, autonomy is evaluated as a result of the functional design, and its trade-offs against fundamental qualities like **Reliability** and **Maintainability** are best analyzed through a decision model, as we will demonstrate in Section 5.3.

5 Decision Model for Architectural Trade-offs

The taxonomy presented in Section 4 provides a structured classification of architectural options that answers RQ1 and RQ2. However, it does not inherently guide an architect on *which* option to choose in a given context to meet specific quality requirements. To bridge this gap between taxonomy and practical design, we introduce a decision model. This model aims to equip architects with a systematic tool to reason about the trade-offs inherent in designing FM-based agents.

5.1 Derivation of the Decision Model

The decision matrix is the result of a **systematic synthesis of the evidence collected from the primary studies** included in our SLR. The derivation process involved the following steps:

- **Evidence Extraction**: During the data extraction phase of our SLR, we not only catalogued the architectural options used in each study but also explicitly extracted any claims, discussions, or empirical results related to the *impact* of those choices on system-level qualities (e.g., statements about performance, reliability, maintainability).
- **Thematic Synthesis**: We then synthesized this extracted evidence. For each functional architectural choice in our taxonomy (e.g., `Multi-path Plan Generator`), we aggregated all associated non-functional impacts reported across the literature.
- **Impact Assessment**: Based on the consensus or prevalence of evidence in the literature, we assigned a qualitative impact assessment (++(strong positive), + (positive), o(neutral), -(negative), --(strong negative)) for each intersection in the matrix. For example, if multiple studies empirically demonstrated or strongly argued that a specific choice significantly enhances a quality, it was marked with ++.

The impact ratings in the Decision Matrix were synthesized through a systematic analysis of the 66 primary studies. For each study, the primary researcher extracted observed design trade-offs and mapped them to non-functional qualities, recording these as qualitative observations in the data extraction repository (e.g., the "Non-functional Qualities" column in our data extraction form). To resolve instances of conflicting evidence, the primary researcher performed a cross-case comparison to identify the most prevalent architectural trends across the dataset. These synthesized ratings were then independently reviewed and cross-checked by a second researcher to ensure consistency and mitigate individual bias. Finally, the matrix was validated against the practical engineering scenarios included in our expert questionnaire (e.g., the "AI Analyst Agent" case), ensuring the model reflects a grounded consensus between synthesized literature evidence and practitioner experience.

5.2 The Architectural Trade-off Decision Matrix

To illustrate how this matrix (Table 4) is used to navigate complex design trade-offs, we now analyze two foundational architectural scenarios drawn from the literature.

Architectural Scenario 1: The Planning Trade-off – Flexibility vs. Predictability This scenario concerns the choice of a plan generation strategy.

- **The Choice**: An architect must decide between a `Single-path Plan Generator` and a `Multi-path Plan Generator`.
- **Positive Consequences**: Opting for a `Multi-path Plan Generator` endows the agent with high adaptability. As shown in our matrix, this greatly enhances **Usability** (++) in dynamic scenarios and improves **Robustness** (++) by allowing the agent to explore multiple solution pathways to overcome unexpected obstacles.
- **Negative Consequences**: However, this flexibility introduces significant non-determinism. This makes the agent's behavior difficult to test and reproduce, which, as the matrix indicates, severely degrades **Maintainability** (--) and its sub-attribute, *testability*.
- **The Resulting Trade-off**: The architect is therefore forced into a classic trade-off: they must sacrifice the engineering predictability and ease of maintenance offered by a single-path approach in order to gain the high adaptability and robustness required for complex environments.

Architectural Scenario 2: The Memory Trade-off – Human-centric vs. Machine-centric Design This scenario addresses the choice of memory format, a critical decision in designing explainable and efficient agents.

- **The Choice**: The decision lies between a human-readable format like `Natural Language Memory` and a machine-optimized format like `Embedding Memory`.
- **Positive Consequences**: Choosing `Natural Language Memory` provides superior transparency. Its inherent human-readability, as reflected in the matrix, is crucial for **Responsible & Trustworthy AI** (++) (specifically, explainability) and **Usability** (++), especially in domains requiring human oversight.
- **Negative Consequences**: This transparency comes at a steep price. Processing unstructured text incurs high computational overhead, severely impacting **Performance** (--). Conversely, `Embedding Memory` offers highly efficient retrieval, boosting **Performance** (++), but its black-box nature is detrimental to both **RAI** (--) and **Usability** (--).
- **The Resulting Trade-off**: The architect must make a conscious decision between prioritizing human-centric qualities like explainability and usability, or machine-centric qualities like computational performance and efficiency.

5.3 A Composite Example: Architecting for Composite Goals

Beyond analyzing the trade-offs of individual architectural choices, the decision model's primary utility lies in reasoning about **composite, real-world design goals**. These goals, such as achieving a high level of autonomy, are often not fundamental categories within the taxonomy itself, but are rather complex characteristics that emerge from the interplay of multiple architectural decisions. A prime example is architecting for a *high level of autonomy*. As we defined in Section 4.12, this is an emergent property. To achieve it, an architect must select a specific *constellation* of functional capabilities. Based on our taxonomy, this would likely involve the synergistic combination of:

- A `Multi-path Plan Generator` to handle unforeseen obstacles.

- `Fully Distributed Coordination` to eliminate single points of failure.
- Advanced `Self-learning` capabilities to adapt without human intervention.

Using our decision matrix, an architect can then analyze the cumulative non-functional consequences of this "high-autonomy" architectural configuration. On one hand, this combination significantly enhances **Reliability** (++) by removing central points of failure and boosts the adaptability aspect of **Usability** (++) in dynamic environments. Conversely, these benefits are counterbalanced by significant drawbacks. The communication overhead inherent in `Fully Distributed Coordination` severely degrades **Performance** (--). The system's resultant complexity and non-determinism present considerable challenges for debugging, thus drastically reducing **Maintainability** (--). Furthermore, ensuring such a highly autonomous system behaves predictably and ethically introduces significant challenges for **Responsible & Trustworthy AI**.

5.4 Illustrative Case Study: AI Audit Analyst

To demonstrate the practical utility and decision-making lifecycle of our model, we present a case study focused on developing an AI Audit Analyst, a system designed to autonomously process large volumes of corporate financial records to ensure regulatory compliance (Figure 6). In this engineering context, the architect identifies two high-priority non-functional requirements: Reliability (the system must not hallucinate audit findings) and Auditability (every decision must be traceable for legal reasons). Conversely, the system has a lower priority for Latency, as audits are typically batch-processed offline.

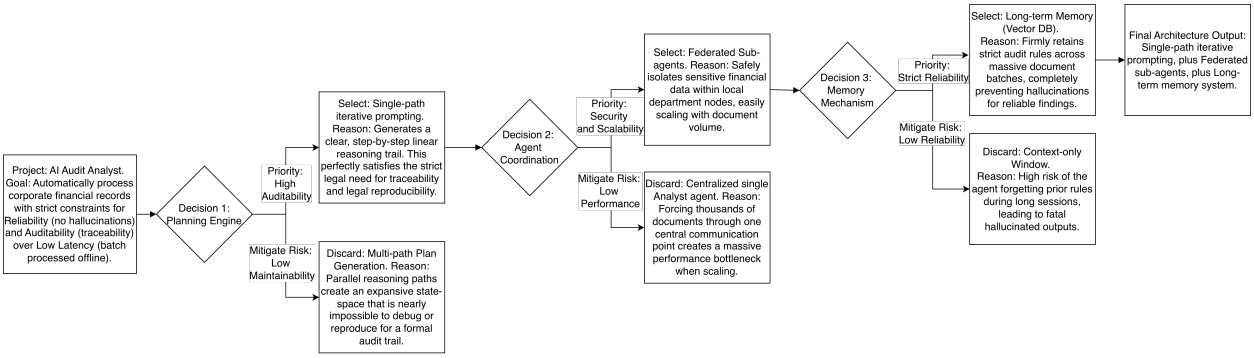


Figure 6: Decision-making workflow for the AI Audit Analyst case study. The flowchart demonstrates the step-by-step traversal from initial non-functional requirements (NFRs) to the final composite architecture, illustrating how the decision matrix systematically resolves architectural conflicts across Planning, Coordination, and Memory dimensions.

Guided by the decision-making workflow, the architect first consults the taxonomy to identify candidate components for the Planning and Coordination dimensions. When evaluating Planning options, the architect faces a trade-off between Single-path and Multi-path reasoning. While Multi-path generators can improve task success rates, our Decision Matrix indicates a significant negative impact on Maintainability. This rating is supported by SLR evidence highlighting that parallel reasoning paths create an expansive state-space that is difficult to debug or reproduce during an audit trail. Prioritizing Auditability, the architect selects a Single-path iterative prompting mechanism with self-verification.

Next, for Agent Coordination, the architect evaluates Centralized versus Federated models. Although a Centralized model is easier to implement, the matrix reveals potential Performance bottlenecks when scaling to thousands of documents. By choosing a Federated approach, the architect achieves better Scalability and Security by isolating sensitive data within sub-agents, as suggested by practitioners in our validation survey.

Finally, to guarantee the strict Reliability constraint over extended audit sessions, the architect evaluates the Memory dimension. While a Context-only approach is simpler, the decision matrix flags a severe risk to Reliability, as the agent might forget prior audit rules when processing large document batches. To prevent hallucinations and ensure the consistent application of standards, the architect selects Long-term Memory (e.g., via a Vector Database), accepting the trade-off of increased architectural Complexity. The resulting composite architecture—Single-path reasoning, Federated coordination, and Long-term memory—demonstrates how this structured traversal—from requirement prioritization to matrix-guided selection—transforms the theoretical framework into a practical handbook for resolving complex architectural conflicts.

6 Validation

A multi-faceted approach was employed to validate the robustness, completeness, and practical applicability of the proposed taxonomy and decision model. The validation relies primarily on a formal survey with a panel of experts, while its structural integrity is supported by the rigorous, evidence-based methodology used for its construction.

6.1 Methodological Groundedness and Structural Integrity

The structural integrity of our taxonomy is established through its rigorous construction process. As detailed in Section 3, the entire study was guided by the systematic protocols for SLRs and the established taxonomy development guidelines from Usman et al. [19], ensuring an empirically grounded foundation.

A primary outcome of this rigorous process was ensuring the taxonomy's **orthogonality**—the logical independence of its dimensions. This was a deliberate design goal, enforced by adhering to the principles of the Usman et al. methodology [19] (specifically steps B8-B11, as discussed in Section 3). This structured approach required a deliberate analysis of category relationships to ensure they were mutually exclusive. The success of this design choice was then formally confirmed through our external expert survey, as detailed in the next section.

6.2 External Validation: Expert-Driven Survey

To assess the taxonomy's quality and its practical relevance from an external perspective, we conducted a formal validation survey with a panel of experts from both industry and academia. We invited 12 experts to participate in the evaluation. Participants were identified through two primary channels: (1) academic experts selected for their scholarly contributions to the field, including authors of some primary studies in our SLR, and (2) senior industry practitioners contacted via professional networks (including LinkedIn and specialized AI engineering communities), selected for their recognized, practical experience in designing and developing complex AI-based systems. The panel comprised 6 academic researchers (including 2 full professors) in Software Engineering and AI, and 6 senior industry practitioners (including 4 software architects and 2 AI/ML engineers) with an average of 5+ years of experience in designing and developing complex AI-based systems. The diverse backgrounds and high level of seniority within the panel ensure that the validation results are representative of both state-of-the-art research and industrial best practices. To ensure methodological rigor, the survey followed standard ethical guidelines: all participants provided informed consent, and their data were fully anonymized. For the analysis of open-ended feedback, we employed thematic analysis, where two researchers independently coded the responses and resolved any discrepancies through a negotiated agreement process. The profiles of these experts are summarized in Table 3.

Table 3: Profiles of Experts in the External Validation Survey

Affiliation	Professional Role	Year	Expertise Level	Count
Academia	Full Professor	15+	Senior/Expert	2
	Senior Researcher	5+	Mid-Senior	4
Industry	Senior Software Architect	10+	Senior/Expert	4
	AI/ML Engineer	5+	Professional	2
Total				**12**

We designed a structured questionnaire (The questionnaire is included in the appendix. The full instrument is available in our replication package) to guide the evaluation. The experts were provided with our manuscript, the main taxonomy figures, and the decision matrix. The questionnaire was divided into two main parts. For the taxonomy, we asked experts to evaluate its completeness, correctness, and orthogonality; for the decision model, we focused on its clarity and practical utility. Feedback was collected through a mix of five-point Likert scale ratings (from 1=Strongly Disagree to 5=Strongly Agree) and open-ended questions.

6.2.1 Results of the Taxonomy Evaluation

The first part of the survey assessed the taxonomy based on three key criteria, and the quantitative results indicate a strong positive consensus from the expert panel. The full survey instrument and detailed results are available in our online replication package.

- **Completeness:** This criterion evaluated whether the taxonomy covers all critical architectural aspects of FM-based agents. The mean score from the 12 experts was 4.63 (out of 5.0), with a standard deviation of 0.53, indicating strong agreement.
- **Correctness:** This assessed whether the dimensions and categories are logically sound and well-defined. The mean score was 4.59, with a standard deviation of 0.66, reflecting a very high level of agreement on the taxonomy's logical integrity.
- **Orthogonality:** While experts affirmed the clear independence of foundational categories such as Input Modality, their comments on more interconnected areas were particularly elucidating. For instance, the discussion around the procedural connection between Planning and Action, and the conceptual proximity of Reliability, Robustness, and Resilience, underscored the value of our taxonomy's deliberate distinctions. Our taxonomy intentionally separates these design options to provide architects with the necessary precision to distinguish high-level strategy from runtime execution. Thus, the feedback confirmed that our approach to orthogonality is not based on creating isolated silos, but on providing the clear and essential distinctions needed for practical architectural reasoning.

6.2.2 Results of the Decision Model Evaluation

The second part of the survey focused on the Decision Matrix, which was also received very positively by the experts, particularly for its practical applicability.

- **Clarity:** This criterion evaluated whether the model clearly illustrates the architectural trade-offs. The mean score was 4.17, with a standard deviation of 0.72, indicating exceptional agreement on its clarity.
- **Utility:** This assessed whether the model is perceived as a practical and useful tool for architects. The mean score was 4.58, with a standard deviation of 0.67, confirming its high practical value.

The qualitative feedback strongly supported the Decision Model's value as a practical tool for architectural reasoning. Experts praised the model for making critical trade-offs "explicit", a feature one participant noted is "often missing in academic frameworks". This clarity was attributed to the matrix format, which makes complex architectural compromises "visible at a glance," thereby enabling more informed and deliberate design choices. The constructive suggestions for future enhancements further affirmed the model's potential. These included exploring ways to quantify the impacts of design choices, extending the model with a `Cost` dimension, and providing common mitigation patterns for negative trade-offs. This feedback confirms the model's role as a practical and extensible foundation for future architectural research.

7 Threats to Validity

We discuss the potential threats to the validity of our study using the well-established four-category framework for software engineering research [101]. We detail each potential threat and the mitigation strategies we employed.

7.1 Construct Validity

Construct validity concerns the extent to which our study measures what it claims to be measuring. A primary threat here is that our search keywords might not have captured all relevant studies on FM-based agent architectures. To mitigate this, we employed a multi-pronged search strategy. We used the Quasi-Gold Standard (QGS) method to systematically derive our search string from a known set of highly relevant papers. Furthermore, we supplemented our keyword search with both forward and backward snowballing to identify relevant studies that may have been missed.

7.2 Internal Validity

Internal validity relates to confounding factors that could influence the study's results, primarily researcher bias. A threat exists in the subjective nature of study selection and data extraction. To mitigate this, we designed a rigorous protocol with several safeguards. We used a clear and objective set of inclusion and exclusion criteria (Table 1). The classification and data extraction process involved an initial classification by one researcher, followed by a complete review and verification by a second, senior researcher. Any disagreements were resolved through consensus-based discussion among all authors to ensure consistency.

A potential threat to internal validity relates to the qualitative coding process used to develop the taxonomy. As described in Section 3.1.5, we employed a negotiated agreement approach rather than independent double-coding. Consequently, we did not calculate an inter-rater reliability statistic such as Cohen's Kappa. While our consensus-driven

approach facilitated deep discussions and resolution of complex architectural nuances, we acknowledge that the lack of independent double-coding represents a limitation in demonstrating strict quantitative reliability of the coding process. This limitation is not confined to taxonomy coding: study classification and data extraction, and the quality assessment scoring in Section 3.2.1, followed the same one-researcher-then-audit process rather than independent double-coding. We acknowledge this as a methodological limitation across our subjective analysis tasks and recommend that future replications incorporate independent double-coding with reliability metrics throughout the pipeline.

The impact ratings in the Decision Matrix are derived from a qualitative synthesis of literature and researcher observations. This introduces a potential risk of subjective bias in how trade-offs were interpreted and mapped. We mitigated this risk by implementing a dual-researcher cross-check protocol and grounding the ratings in the "Non-functional Qualities" evidence recorded during data extraction. Furthermore, the expert survey served as an external sanity check; however, we acknowledge that the survey focused on high-level consensus rather than a granular validation of every individual cell in the matrix.

We acknowledge that the exclusion of papers by GPT-4o was not independently supervised at the time. The use of GPT-4o during screening may have introduced false exclusions or systematic bias. Although the primary researcher reviewed the filtering results at each round and retained responsibility for all final decisions, we did not conduct a separate sampling audit of the excluded papers to estimate agreement with an independent manual assessment. We acknowledge this as a limitation and recommend that future reviews using LLM-assisted screening include a documented random audit of excluded studies.

7.3 External Validity

External validity concerns the generalizability of our findings. We identify four primary threats here:

- **Reliance on Non-Peer-Reviewed Pre-prints:** This is a major threat, as a significant portion of our primary studies are pre-prints from sources like arXiv, which do not undergo formal peer review. This was a deliberate methodological choice. In a fast-evolving field like FM-based agents, strictly limiting our review to peer-reviewed literature would have resulted in an outdated and less relevant taxonomy. To mitigate the risk of including low-quality work, we implemented a stringent, differentiated **quality assessment (QA) protocol** (detailed in Section 3.2.1), which ensured all included studies, regardless of publication status, met a high and consistent standard of research quality.

  A potential limitation of our differentiated scoring approach is that QAQ9 was assessed only for peer-reviewed studies. Consequently, a peer-reviewed paper published in a less established venue could receive a lower QAQ9 score, whereas this venue-related dimension was excluded from the normalised score of a pre-print. Although this design avoids treating an incomplete publication process as evidence of low methodological quality, it may introduce some imbalance between the two assessment schemes. We mitigated this concern by applying comparable proportional thresholds to the quality dimensions applicable to each publication type and by clearly distinguishing pre-prints from peer-reviewed studies throughout the analysis. In practice, this means a pre-print is not penalised on this dimension in the way a peer-reviewed paper from a less established venue is, effectively placing it on par with a peer-reviewed paper from a reputable venue on this specific dimension.

- **Exclusion of Grey Literature:** As our SLR focused predominantly on academic literature, it explicitly excluded grey literature such as GitHub repositories, Hugging Face model cards, and engineering blogs. Consequently, our taxonomy may better represent the architectural patterns of academic research agents rather than those deployed in real-world industrial settings. We acknowledge this as a limitation and identify the incorporation of these practical sources as a critical avenue for future work.

- **Limited Scope and Domain Coverage of the Expert Validation:** Our expert validation was conducted with a panel of 12 experts. Although the panel was selected to provide a balanced academic and industry perspective, their views may not represent the wider software architecture and AI community. The scores reported in Section 6.2 should therefore be interpreted as agreement within this panel, rather than as evidence that the taxonomy and Decision Matrix apply equally well across all application domains. Highly specialised domains, such as safety-critical embedded systems and medical devices, are underrepresented both in our 66 primary studies and in the backgrounds of the panel. The applicability of the Decision Matrix may therefore vary in contexts with project constraints or technologies not fully represented in our evidence base. We also did not conduct other forms of validation, such as surveying the authors of the primary studies. We acknowledge these limitations and identify broader and domain-stratified validation as directions for future work.

- **Temporal Validity and Taxonomy Evolution:** The rapid pace of FM advancements poses a threat to the long-term relevance of any static taxonomy. To mitigate this, we designed our taxonomy with a hierarchical structure that ensures extensibility. This allows new architectural patterns or emerging capabilities (e.g., multi-modal reasoning) to be integrated as new leaf nodes without restructuring the core dimensions. Furthermore, we acknowledge that the architectural trade-offs captured in our Decision Model are subject to contextual constraints; the impact of specific components on system latency or reliability may vary depending on the underlying model's efficiency and the specific application domain, ensuring the framework remains adaptable as the field evolves.

7.4 Conclusion Validity

Conclusion validity concerns the degree to which conclusions about relationships in our data are reasonable. A threat here is the potential for subjectivity in the thematic analysis and synthesis process used to create the taxonomy from the extracted data. To mitigate this, the terminology control and the grouping of concepts into categories and dimensions were not performed by a single researcher. It was a collaborative process involving multiple rounds of discussion and consensus-building among the authors to ensure the resulting taxonomy structure was a logical and defensible synthesis of the evidence from the literature. This threat also extends to our Decision Model. The model is intended as a high-level reasoning tool for making architectural trade-offs explicit, rather than as a precise quantitative predictor. Synthesising qualitative evidence from multiple studies into impact ratings necessarily involves interpretation and may introduce researcher bias. The model is also constrained by the scope and limitations of the primary literature from which it was derived. Accordingly, the ratings should be interpreted as generalised baselines. Where the primary studies reported mixed results for an architectural option, the assigned rating reflects the most prevalent trend across the dataset, as described in Section 5.1. It therefore indicates the likely direction of a trade-off rather than its magnitude in a specific system. In such borderline cases, architects should use the rating as a starting point for further analysis against their own project constraints, rather than as a definitive answer. Quantifying these impacts, as also suggested by participants in our validation survey, remains an important direction for future work.

8 Discussion

In this section, we discuss the potential implications of our taxonomy and decision model for three key stakeholder groups: academic researchers, software developers and architects, and foundation model (FM) vendors and tool builders. Through these perspectives, we explicitly revisit and synthesize our findings for RQ1 (Architectural Elements), RQ2 (Trade-offs), and RQ3 (Design Guidance).

8.1 Implications for Researchers

For academic researchers, our work offers several benefits. First, the **taxonomy provides a common vocabulary and a structured intellectual framework** for the community. It can be used to classify new agent architectures, enabling clearer comparisons and facilitating a more systematic accumulation of knowledge. Second, by mapping the current landscape, our work helps to **identify under-explored areas and potential research gaps**. For instance, certain combinations of architectural options may be rare in the literature, suggesting avenues for novel research. Finally, our **Decision Model serves as a set of falsifiable hypotheses** about the relationships between architectural choices and system qualities, which can be validated, refined, or challenged in future empirical studies. Specifically, researchers can use the taxonomy dimensions as a basis for defining 'control groups' when benchmarking the performance of different agent architectures.

Takeaway 1 (RQ1): Our taxonomy provides a multi-dimensional framework that formalizes the design space of FM-based agents. It transitions the field from ad-hoc prompting to a systematic "Architectural Engineering" paradigm, offering a common vocabulary for future empirical research.

8.2 Implications for Developers and Architects

For practitioners responsible for building agent-based systems, our contributions serve as a practical guide. The **taxonomy acts as a design space map or a "pattern catalogue,"** presenting architects with a clear and comprehensive set of available architectural options. This can prevent ad-hoc design and encourage a more structured design process. More importantly, the **Decision Model is a practical tool for architectural reasoning**. It forces a consideration of non-functional qualities early in the design process and makes the inherent trade-offs explicit (e.g., choosing a 'Multi-path Plan Generator' for higher adaptability may decrease 'Maintainability'). This allows development teams to make more informed, deliberate design decisions that are consciously aligned with their specific system requirements. In

practice, architects can utilize the Decision Model as a 'pre-flight checklist' during design reviews to systematically evaluate whether the selected components (e.g., Memory, Planning) meet the project's specific latency and reliability constraints.

Takeaway 2 (RQ2): Architectural decisions are characterized by inherent trade-offs (e.g., Autonomy vs. Maintainability). Our Decision Model provides a structured way to navigate these compromises based on specific system-level quality requirements.

8.3 Implications for FM Vendors and Tool Builders

Our work also has direct implications for the vendors who create foundational models and the builders of popular agent frameworks. The taxonomy provides them with a **structured view of how their technologies are being used** in real-world agent architectures. By seeing which architectural options are most frequently combined, they can identify common use-case patterns and prioritize the development of features, APIs, or components that better support these patterns. For example, understanding the trade-offs in different 'Memory' options can guide a tool builder in designing a more versatile and efficient memory module. Ultimately, our framework can help align the evolution of the underlying platforms and tools with the practical needs of architects and developers. For instance, tool builders can use the identified 'rare combinations' in our taxonomy to discover underserved architectural patterns, guiding the development of new standardized APIs or middleware that simplify the integration of complex reasoning loops.

Takeaway 3 (RQ3): The transition from ad-hoc integration to structured design is facilitated by our framework, which acts as a practical guidance for building next-generation, standardized AI agent platforms.

9 Conclusion

FM-based agents are gaining increasing attention in various domains. Our study reveals that the development of these agents is transitioning from ad-hoc experimentation to a structured architectural discipline. By formalizing this space, our taxonomy and decision model extend fundamental software engineering principles—specifically the "Separation of Concerns"—to the stochastic nature of AI-native systems. This taxonomy provides a shared vocabulary that not only addresses standardization issues but also offers a theoretical bridge for applying established design rigor to foundation-model-based architectures. To ensure its ongoing relevance, the taxonomy is designed as an extensible 'living framework', capable of evolving alongside advancements in foundation models.

In future work, we will explore how to apply this taxonomy in conjunction with existing architectural patterns. This study, anchored in a systematic literature review and expert validation, provides a foundational framework. However, as the field of FM-based agents is rapidly evolving, we see significant value in extending and further validating this research through a wider range of empirical methods. Future studies could involve more practitioner-oriented approaches, such as in-depth interviews with industry architects to refine the taxonomy's practical relevance and large-scale mining studies of open-source agent repositories. Further validation could also be achieved by surveying the authors of our primary studies for their direct feedback and by replicating our SLR process with a specific focus on high-quality grey literature to broaden the evidence base. Given the limited size and domain coverage of our validation panel, future work should conduct domain-stratified validation with practitioners from distinct and highly specialised application domains to assess how well the taxonomy and Decision Matrix generalise beyond the contexts covered in this study. We believe such future work will build upon the foundational classification we have established and further bridge the gap between architectural theory and practice.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process: During the preparation of this work the author(s) used GPT-4o in order to assist the SLR process. After using this tool/service, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the content of the published article.

References

[1] Q. Lu, L. Zhu, X. Xu, Z. Xing, S. Harrer, J. Whittle, Towards responsible generative ai: A reference architecture for designing foundation model based agents, in: 2024 IEEE 21st International Conference on Software Architecture Companion (ICSA-C), IEEE, 2024, pp. 119–126.

[2] L. C. Budler, L. Gosak, G. Stiglic, Review of artificial intelligence-based question-answering systems in healthcare, Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery 13 (2) (2023) e1487.

[3] T. Masterman, S. Besen, M. Sawtell, A. Chao, The landscape of emerging ai agent architectures for reasoning, planning, and tool calling: A survey, arXiv preprint arXiv:2404.11584 (2024).

[4] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, A. Anandkumar, Voyager: An open-ended embodied agent with large language models, arXiv preprint arXiv:2305.16291 (2023).

[5] Q. Wu, G. Bansal, J. Zhang, Y. Wu, S. Zhang, E. Zhu, B. Li, L. Jiang, X. Zhang, C. Wang, Autogen: Enabling next-gen llm applications via multi-agent conversation framework, arXiv preprint arXiv:2308.08155 (2023).

[6] A. Jansen, J. Bosch, Software architecture as a set of architectural design decisions, in: 5th Working IEEE/IFIP Conference on Software Architecture (WICSA'05), IEEE, 2005, pp. 109–120.

[7] Google DeepMind, Gemini, `https://deepmind.google/models/gemini/`, accessed: 2026-07-30 (2026).

[8] Meta AI, The llama 4 herd: The beginning of a new era of natively multimodal ai innovation, `https://ai.meta.com/blog/llama-4-multimodal-intelligence/`, accessed: 2026-07-30 (2025).

[9] Taskade, Top 11 open-source autonomous agents & frameworks: The future of self-running ai, accessed: 2024-05-28 (2024).
URL `https://www.taskade.com/blog/top-autonomous-agents/`

[10] S. Hong, X. Zheng, J. Chen, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, et al., Metagpt: Meta programming for multi-agent collaborative framework, arXiv preprint arXiv:2308.00352 (2023).

[11] CrewAI, Crewai, accessed: 2024-05-28 (2024).
URL `https://www.crewai.com/`

[12] K. Mei, Z. Li, S. Xu, R. Ye, Y. Ge, Y. Zhang, Aios: Llm agent operating system, arXiv preprint arXiv:2403.16971 (2024).

[13] Z. Zhao, F. Zhao, Y. Zhao, Y. Zeng, Y. Sun, A brain-inspired theory of mind spiking neural network improves multi-agent cooperation and competition, Patterns 4 (8) (2023).

[14] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, et al., A survey on large language model based autonomous agents, Frontiers of Computer Science 18 (6) (2024) 186345.

[15] S. Schulhoff, M. Ilie, N. Balepur, K. Kahadze, A. Liu, C. Si, Y. Li, A. Gupta, H. Han, S. Schulhoff, et al., The prompt report: A systematic survey of prompting techniques, arXiv preprint arXiv:2406.06608 (2024).

[16] T. Guo, X. Chen, Y. Wang, R. Chang, S. Pei, N. V. Chawla, O. Wiest, X. Zhang, Large language model based multi-agents: A survey of progress and challenges, arXiv preprint arXiv:2402.01680 (2024).

[17] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, M. Zhang, J. Wang, S. Jin, E. Zhou, et al., The rise and potential of large language model based agents: A survey, arXiv preprint arXiv:2309.07864 (2023).

[18] X. Li, A survey on llm-based agents: Common workflows and reusable llm-profiled components, arXiv preprint arXiv:2406.05804 (2024).

[19] M. Usman, R. Britto, J. Börstler, E. Mendes, Taxonomies in software engineering: A systematic mapping study and a revised taxonomy development method, Information and Software Technology 85 (2017) 43–59.

[20] H. Zhang, M. A. Babar, P. Tell, Identifying relevant studies in software engineering, Information and Software Technology 53 (6) (2011) 625–637, special Section: Best papers from the APSEC. `doi:https://doi.org/10.1016/j.infsof.2010.12.010`.
URL `https://www.sciencedirect.com/science/article/pii/S0950584910002260`

[21] B. H. Kwasnik, The role of classification in knowledge representation and discovery (1999).

[22] R. Prieto-Diaz, Implementing faceted classification for software reuse, Communications of the ACM 34 (5) (1991) 88–97.

[23] R. L. Glass, I. Vessey, Contemporary application-domain taxonomies, IEEE software 12 (4) (1995) 63–76.

[24] G. R. Wheaton, Development of a taxonomy of human performance: A review of classificatory systems relating (1968).

[25] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, H. Wang, Large language models for software engineering: A systematic literature review, ACM Transactions on Software Engineering and Methodology 33 (8) (2024) 1–79.

[26] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, I. Polosukhin, Attention is all you need, Advances in neural information processing systems 30 (2017).

[27] H. Naveed, C. Arora, H. Khalajzadeh, J. Grundy, O. Haggag, Model driven engineering for machine learning components: A systematic literature review, Information and Software Technology 169 (2024) 107423.

[28] S. Wang, L. Huang, A. Gao, J. Ge, T. Zhang, H. Feng, I. Satyarth, M. Li, H. Zhang, V. Ng, Machine/deep learning for software engineering: A systematic literature review, IEEE Transactions on Software Engineering 49 (3) (2022) 1188–1231.

[29] International Organization for Standardization, Geneva, Switzerland, ISO/IEC 25010:2023 Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — Product Quality Model, 2nd Edition (2023).

[30] L. Bass, P. Clements, R. Kazman, Software Architecture in Practice, 4th Edition, Addison-Wesley Professional, Boston, MA, 2021.

[31] L. Bass, Q. Lu, I. Weber, L. Zhu, Engineering AI Systems: Architecture and DevOps Essentials, Addison-Wesley Professional, Boston, MA, 2025.

[32] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, et al., Llama: Open and efficient foundation language models, arXiv preprint arXiv:2302.13971 (2023).

[33] Y. Xia, M. Shenoy, N. Jazdi, M. Weyrich, Towards autonomous system: flexible modular production system enhanced with large language model agents, arXiv preprint arXiv:2304.14721 (2023).

[34] C.-C. Chang, C.-T. Ho, Y. Li, Y. Chen, H. Ren, Drc-coder: Automated drc checker code generation using llm autonomous agent, in: Proceedings of the 2025 International Symposium on Physical Design, 2025, pp. 143–151.

[35] K. You, H. Zhang, E. Schoop, F. Weers, A. Swearngin, J. Nichols, Y. Yang, Z. Gan, Ferret-ui: Grounded mobile ui understanding with multimodal llms, in: European Conference on Computer Vision, Springer, 2025, pp. 240–255.

[36] B. Xu, X. Shu, H. Mei, G. Xie, B. Fernando, M. Z. Shou, J. Tang, Fedmllm: Federated fine-tuning mllm on multimodal heterogeneity data, arXiv preprint arXiv:2411.14717 (2024).

[37] D. Zhao, Z. Xing, X. Xia, D. Ye, X. Xu, L. Zhu, Seehow: Workflow extraction from programming screencasts through action-aware video analytics, arXiv preprint arXiv:2304.14042 (2023).

[38] J. Yang, S. Gao, Y. Qiu, L. Chen, T. Li, B. Dai, K. Chitta, P. Wu, J. Zeng, P. Luo, et al., Generalized predictive model for autonomous driving, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2024, pp. 14662–14672.

[39] A. Modarressi, A. Imani, M. Fayyaz, H. Schütze, Ret-llm: Towards a general read-write memory for large language models, arXiv preprint arXiv:2305.14322 (2023).

[40] Y. Gu, Y. Shu, H. Yu, X. Liu, Y. Dong, J. Tang, J. Srinivasa, H. Latapie, Y. Su, Middleware for llms: Tools are instrumental for language agents in complex environments, arXiv preprint arXiv:2402.14672 (2024).

[41] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao, React: Synergizing reasoning and acting in language models, arXiv preprint arXiv:2210.03629 (2022).

[42] S. Gandhi, M. Patwardhan, L. Vig, G. Shroff, Budgetmlagent: A cost-effective llm multi-agent system for automating machine learning tasks, in: Proceedings of the 4th International Conference on AI-ML Systems, 2024, pp. 1–9.

[43] M. A. Islam, M. E. Ali, M. R. Parvez, Mapcoder: Multi-agent code generation for competitive problem solving, arXiv preprint arXiv:2405.11403 (2024).

[44] M. Mostajabdaveh, T. T. Yu, R. Ramamonjison, G. Carenini, Z. Zhou, Y. Zhang, Optimization modeling and verification from problem specifications using a multi-agent multi-stage llm framework, INFOR: Information Systems and Operational Research 62 (4) (2024) 599–617.

[45] H. Takagi, S. Moriya, T. Sato, M. Nagao, K. Higuchi, A framework for efficient development and debugging of role-playing agents with large language models, in: Proceedings of the 30th International Conference on Intelligent User Interfaces, 2025, pp. 70–88.

[46] Y. Xu, F. Lin, J. Yang, N. Tsantalis, et al., Mantra: Enhancing automated method-level refactoring with contextual rag and multi-agent llm collaboration, arXiv preprint arXiv:2503.14340 (2025).

[47] W. Chen, Y. Su, J. Zuo, C. Yang, C. Yuan, C.-M. Chan, H. Yu, Y. Lu, Y.-H. Hung, C. Qian, et al., Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors, in: The Twelfth International Conference on Learning Representations, 2023.

[48] C. Qian, X. Cong, C. Yang, W. Chen, Y. Su, J. Xu, Z. Liu, M. Sun, Communicative agents for software development, arXiv preprint arXiv:2307.07924 (2023).

[49] K. Zhang, J. Li, G. Li, X. Shi, Z. Jin, Codeagent: Enhancing code generation with tool-integrated agent systems for real-world repo-level coding challenges, arXiv preprint arXiv:2401.07339 (2024).

[50] X. Liu, X. Lou, J. Jiao, J. Zhang, Position: Foundation agents as the paradigm shift for decision making, arXiv preprint arXiv:2405.17009 (2024).

[51] S. Hao, Y. Gu, H. Ma, J. J. Hong, Z. Wang, D. Z. Wang, Z. Hu, Reasoning with language model is planning with world model, arXiv preprint arXiv:2305.14992 (2023).

[52] L. Guan, K. Valmeekam, S. Sreedharan, S. Kambhampati, Leveraging pre-trained large language models to construct and utilize world models for model-based task planning, Advances in Neural Information Processing Systems 36 (2023) 79081–79094.

[53] K. Valmeekam, K. Stechly, S. Kambhampati, Llms still can't plan; can lrms? a preliminary evaluation of openai's o1 on planbench, URL https://arxiv. org/abs/2409.13373 (2024).

[54] C. Qu, S. Dai, X. Wei, H. Cai, S. Wang, D. Yin, J. Xu, J.-R. Wen, Tool learning with large language models: A survey, arXiv preprint arXiv:2405.17935 (2024).

[55] Y. Ye, X. Cong, S. Tian, J. Cao, H. Wang, Y. Qin, Y. Lu, H. Yu, H. Wang, Y. Lin, et al., Proagent: From robotic process automation to agentic process automation, arXiv preprint arXiv:2311.10751 (2023).

[56] Z. Wang, Z. Liu, Y. Zhang, A. Zhong, L. Fan, L. Wu, Q. Wen, Rcagent: Cloud root cause analysis by autonomous agents with tool-augmented large language models, arXiv preprint arXiv:2310.16340 (2023).

[57] W. Zhou, Y. E. Jiang, L. Li, J. Wu, T. Wang, S. Qiu, J. Zhang, J. Chen, R. Wu, S. Wang, et al., Agents: An open-source framework for autonomous language agents, arXiv preprint arXiv:2309.03409 (2023).

[58] J. Wang, J. Wang, B. Athiwaratkun, C. Zhang, J. Zou, Mixture-of-agents enhances large language model capabilities, arXiv preprint arXiv:2406.04692 (2024).

[59] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, D. Zhou, Self-consistency improves chain of thought reasoning in language models, arXiv preprint arXiv:2203.11171 (2022).

[60] S. Yao, D. Yu, J. Zhao, I. Shafran, T. Griffiths, Y. Cao, K. Narasimhan, Tree of thoughts: Deliberate problem solving with large language models, Advances in Neural Information Processing Systems 36 (2024).

[61] A. Maharana, D.-H. Lee, S. Tulyakov, M. Bansal, F. Barbieri, Y. Fang, Evaluating very long-term conversational memory of llm agents, arXiv preprint arXiv:2402.17753 (2024).

[62] J. S. Park, J. O'Brien, C. J. Cai, M. R. Morris, P. Liang, M. S. Bernstein, Generative agents: Interactive simulacra of human behavior, in: Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology, 2023, pp. 1–22.

[63] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, S. Yao, Reflexion: Language agents with verbal reinforcement learning, Advances in Neural Information Processing Systems 36 (2024).

[64] W. Zhong, L. Guo, Q. Gao, H. Ye, Y. Wang, Memorybank: Enhancing large language models with long-term memory, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 38, 2024, pp. 19724–19731.

[65] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat, et al., Gpt-4 technical report, arXiv preprint arXiv:2303.08774 (2023).

[66] Z. Zhang, X. Bo, C. Ma, R. Li, X. Chen, Q. Dai, J. Zhu, Z. Dong, J.-R. Wen, A survey on the memory mechanism of large language model based agents, arXiv preprint arXiv:2404.13501 (2024).

[67] H. Gao, Y. Zhang, Memory sharing for large language model based agents, arXiv preprint arXiv:2404.09982 (2024).

[68] S. Zhang, Z. Xing, R. Guo, F. Xu, L. Chen, Z. Zhang, X. Zhang, Z. Feng, Z. Zhuang, Empowering agile-based generative software development through human-ai teamwork, ACM Transactions on Software Engineering and Methodology (2025).

[69] C. Zhu, M. Dastani, S. Wang, A survey of multi-agent deep reinforcement learning with communication, Autonomous Agents and Multi-Agent Systems 38 (1) (2024) 4.

[70] S. Agashe, Y. Fan, X. E. Wang, Evaluating multi-agent coordination abilities in large language models, arXiv preprint arXiv:2310.03903 (2023).

[71] H. Du, S. Thudumu, R. Vasa, K. Mouzakis, A survey on context-aware multi-agent systems: Techniques, challenges and future directions, arXiv preprint arXiv:2402.01968 (2024).

[72] E. Pesce, G. Montana, Learning multi-agent coordination through connectivity-driven communication, Machine Learning 112 (2) (2023) 483–514.

[73] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, T. Scialom, Toolformer: Language models can teach themselves to use tools, Advances in Neural Information Processing Systems 36 (2023) 68539–68551.

[74] X. Deng, Y. Gu, B. Zheng, S. Chen, S. Stevens, B. Wang, H. Sun, Y. Su, Mind2web: Towards a generalist agent for the web, Advances in Neural Information Processing Systems 36 (2023) 28091–28114.

[75] S. Qiao, H. Gui, C. Lv, Q. Jia, H. Chen, N. Zhang, Making language models better tool learners with execution feedback, arXiv preprint arXiv:2305.13068 (2023).

[76] Z. Durante, R. Gong, B. Sarkar, N. Wake, R. Taori, P. Tang, S. Lakshmikanth, K. Schulman, A. Milstein, H. Vo, et al., An interactive agent foundation model, in: Proceedings of the Computer Vision and Pattern Recognition Conference, 2025, pp. 3652–3662.

[77] S. Feng, C. Chen, Prompting is all you need: Automated android bug replay with large language models, in: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, 2024, pp. 1–13.

[78] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, K. Narasimhan, Swe-bench: Can language models resolve real-world github issues?, arXiv preprint arXiv:2310.06770 (2023).

[79] G. Baechler, S. Sunkara, M. Wang, F. Zubach, H. Mansoor, V. Etter, V. Cărbune, J. Lin, J. Chen, A. Sharma, Screenai: A vision-language model for ui and infographics understanding, arXiv preprint arXiv:2402.04615 (2024).

[80] C. Zhang, S. He, J. Qian, B. Li, L. Li, S. Qin, Y. Kang, M. Ma, Q. Lin, S. Rajmohan, et al., Large language model-brained gui agents: A survey, arXiv preprint arXiv:2411.18279 (2024).

[81] L. Brand, B. G. Humm, A. Krajewski, A. Zender, Towards improved user experience for artificial intelligence systems, in: International Conference on Engineering Applications of Neural Networks, Springer, 2023, pp. 33–44.

[82] G. Research, Empowering users in digital privacy management through interactive llm-based agents, arXiv preprint arXiv:2410.11906 (2024).

[83] A. Chan, R. Salganik, A. Markelius, C. Pang, N. Rajkumar, D. Krasheninnikov, L. Langosco, Z. He, Y. Duan, M. Carroll, et al., Harms from increasingly agentic algorithmic systems (2023).

[84] T. Gupta, W. Gong, C. Ma, N. Pawlowski, A. Hilmkil, M. Scetbon, M. Rigter, A. Famoti, A. J. Llorens, J. Gao, et al., The essential role of causality in foundation world models for embodied ai, arXiv preprint arXiv:2402.06665 (2024).

[85] S. Yang, O. Nachum, Y. Du, J. Wei, P. Abbeel, D. Schuurmans, Foundation models for decision making: Problems, methods, and opportunities, arXiv preprint arXiv:2303.04129 (2023).

[86] K. Huang, J. Zhang, X. Bao, X. Wang, Y. Liu, Comprehensive fine-tuning large language models of code for automated program repair, IEEE Transactions on Software Engineering (2025).

[87] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, et al., Agentbench: Evaluating llms as agents, arXiv preprint arXiv:2308.03688 (2023).

[88] L. Longo, M. Brcic, F. Cabitza, J. Choi, R. Confalonieri, J. Del Ser, R. Guidotti, Y. Hayashi, F. Herrera, A. Holzinger, et al., Explainable artificial intelligence (xai) 2.0: A manifesto of open challenges and interdisciplinary research directions, Information Fusion 106 (2024) 102301.

[89] T. R. Sumers, S. Yao, K. Narasimhan, T. L. Griffiths, Cognitive architectures for language agents, arXiv preprint arXiv:2309.02427 (2023).

[90] M. Biagiola, P. Tonella, Testing of deep reinforcement learning agents with surrogate models, ACM Transactions on Software Engineering and Methodology 33 (3) (2024) 1–33.

[91] S. Hao, S. Sukhbaatar, D. Su, X. Li, Z. Hu, J. Weston, Y. Tian, Training large language models to reason in a continuous latent space, arXiv preprint arXiv:2412.06769 (2024).

[92] S. Peng, X. Hu, Q. Yi, R. Zhang, J. Guo, D. Huang, Z. Tian, R. Chen, Z. Du, Q. Guo, et al., Self-driven grounding: Large language model agents with automatical language-aligned skill learning, arXiv preprint arXiv:2309.01352 (2023).

[93] Z. Wu, C. Han, Z. Ding, Z. Weng, Z. Liu, S. Yao, T. Yu, L. Kong, Os-copilot: Towards generalist computer agents with self-improvement, arXiv preprint arXiv:2402.07456 (2024).

[94] R. Kamoi, Y. Zhang, N. Zhang, J. Han, R. Zhang, When can llms actually correct their own mistakes? a critical survey of self-correction of llms, Transactions of the Association for Computational Linguistics 12 (2024) 1417–1440.

[95] S. Wu, O. Irsoy, S. Lu, V. Dabravolski, M. Dredze, S. Gehrmann, P. Kambadur, D. Rosenberg, G. Mann, Bloomberggpt: A large language model for finance, arXiv preprint arXiv:2303.17564 (2023).

[96] L. Yi, H. Yu, C. Ren, H. Zhang, G. Wang, X. Liu, X. Li, Fedmoe: Data-level personalization with mixture of experts for model-heterogeneous personalized federated learning, arXiv preprint arXiv:2402.01350 (2024).

[97] K. Lu, H. Yuan, R. Lin, J. Lin, Z. Yuan, C. Zhou, J. Zhou, Routing to the expert: Efficient reward-guided ensemble of large language models, arXiv preprint arXiv:2311.08692 (2023).

[98] X. Wu, S.-h. Wu, J. Wu, L. Feng, K. C. Tan, Evolutionary computation in the era of large language model: Survey and roadmap, arXiv preprint arXiv:2401.10034 (2024).

[99] Y. Kang, T. Fan, H. Gu, L. Fan, Q. Yang, Grounding foundation models through federated transfer learning: A general framework, arXiv preprint arXiv:2311.17431 (2023).

[100] A. Mabrouk, R. P. D. Redondo, M. Abd Elaziz, M. Kayed, Ensemble federated learning: An approach for collaborative pneumonia diagnosis, Applied Soft Computing 144 (2023) 110500.

[101] P. Lago, P. Runeson, Q. Song, R. Verdecchia, Threats to validity in software engineering–hypocritical paper section or essential analysis?, in: Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement, 2024, pp. 314–324.

A Questionnaire and Tables

Expert Evaluation of a Taxonomy and Decision Model for Foundation Model-based Agent Architectures

Part 1: Introduction & Instructions

Thank you for dedicating your valuable time and expertise to this study. Our research addresses the growing architectural challenges in designing and operating systems that leverage Foundation Model-based (FM-based) agents. Given the rapid innovation in this field, there is a significant need for a structured framework to guide architects and developers. To fill this gap, our work introduces a comprehensive taxonomy of architectural options for FM-based agents and an accompanying decision model to navigate design trade-offs. Your expert feedback is crucial for validating the rigor, completeness, and utility of our work. The goal of this evaluation is to assess our proposed taxonomy and decision model against several key criteria:

- **Taxonomy Evaluation Objectives:**
  - Completeness: Does the taxonomy cover the essential architectural dimensions and options for modern FM-based agents?
  - Correctness - Clarity & Structure: Are the categories clear, well-defined, and logically organized?
  - Orthogonality: Are the primary dimensions and categories conceptually independent, minimizing ambiguity and overlap?
- **Decision Model Evaluation Objectives:**
  - Clarity: Does the decision model clearly illustrate the trade-offs between architectural choices and their impact on system qualities?
  - Utility: Is the decision model a practical and useful tool for architects during the design process?

To ensure a smooth evaluation process, please follow these steps:

1. **Review the Core Document:** First, please carefully review the accompanying **"Expert Review Document,"** which presents the complete taxonomy and the Architectural Trade-off Decision Matrix. This document provides the necessary context for all the questions that follow.
2. **Complete the Questionnaire:** After reviewing the document, please proceed to answer the questions in each section of this questionnaire. Your detailed feedback, especially in the open-ended questions, is highly appreciated.
3. **Confidentiality:** Your responses will be kept strictly confidential and used for academic research purposes only. All published results will be anonymized.

We estimate this evaluation will take approximately **20-40 minutes** to complete. We deeply appreciate your contribution to advancing the field of software architecture for AI systems.

Part 2: Overall Assessment of the Taxonomy

This section asks for your overall impression of the *Functional Capabilities* and *Non-functional Qualities* dimensions of the taxonomy.

2.1. Functional Capabilities Taxonomy (Completeness)

a. On a scale of 1 (Strongly Disagree) to 5 (Strongly Agree), please rate your agreement with the following statement: "The Functional Capabilities taxonomy is comprehensive, covering the essential architectural aspects of modern FM-based agents."

[ ] 1 [ ] 2 [ ] 3 [ ] 4 [ ] 5

b. Are there any major functional categories or architectural options you believe are missing? For example, are there any emerging trends from recent research or popular open-source agent frameworks that are not yet captured?

(Please type your answer here)

2.2. Feedback on Functional Capabilities (Clarity & Structure)

a. On a scale of 1 (Very Unclear) to 5 (Very Clear), please rate the overall clarity and structure of the Functional Capabilities taxonomy.

[ ] 1 [ ] 2 [ ] 3 [ ] 4 [ ] 5

b. Are there any existing functional categories that are unclear, should be renamed, or could be merged with others?

(Please type your answer here)

2.3. Non-functional Qualities Taxonomy (Completeness)

a. On a scale of 1 (Strongly Disagree) to 5 (Strongly Agree), please rate your agreement with the following statement: "The Non-functional Qualities taxonomy covers the critical quality attributes for designing and evaluating FM-based agents."

[ ] 1 [ ] 2 [ ] 3 [ ] 4 [ ] 5

b. Are there any critical non-functional qualities you believe are missing? (e.g., considerations for cost, ethics, interoperability, etc.) For example, are there any emerging trends from recent research or popular open-source agent frameworks that are not yet captured?

(Please type your answer here)

2.4. Feedback on Non-functional Qualities (Clarity & Structure)

a. On a scale of 1 (Very Unclear) to 5 (Very Clear), please rate the overall clarity and structure of the Non-functional Qualities taxonomy.

[ ] 1 [ ] 2 [ ] 3 [ ] 4 [ ] 5

b. Are there any existing non-functional categories that are unclear, should be renamed, or could be merged with others?

(Please type your answer here)

Part 3: Evaluation of Orthogonality

A fundamental principle of a robust taxonomy is orthogonality, meaning its categories should be conceptually independent to avoid ambiguity and ensure clear classification. This section is designed to rigorously test this property within our taxonomy. We have found that evaluating the independence of conceptually adjacent categories is a particularly effective method for identifying potential structural weaknesses.

Instructions:

For each taxonomy category listed in the first column below (9 from Functional Capabilities and 9 from Non-functional Qualities), please consider its definition within our taxonomy. In the second column, please identify any other categories from the full list that you believe it conceptually overlaps with. In the third column, please provide a brief rationale for why you perceive an overlap. If you see no significant overlap for a category, you may leave the corresponding row blank.

Taxonomy Category	**Potential Overlapping Categories**	**Rationale for your rating (Optional but helpful)**
Functional Capabilities		
Input Modality	*(Please type your answer here)*	*(Please type your answer here)*
Context		
Role-playing		
Goal Type		
Planning		
Memory		
Action		
Reflection		
Learning Capability		
Workflow		

Access to Underlying Models		
Non-functional Qualities		
Performance		
Reliability		
Robustness		
Resilience		
Security		
Maintainability		
Responsible & Trustworthy AI		
Usability		
Compatibility		

Part 4: Evaluation of the Decision Model

This section evaluates the clarity and practical utility of the **Trade-off Decision Matrix**.

4.1. Clarity of the Decision Model

a. On a scale of 1 (Very Unclear) to 5 (Very Clear), how clear is the Decision Matrix in illustrating the trade-offs between functional choices and their impact on non-functional qualities?

[ ] 1 [ ] 2 [ ] 3 [ ] 4 [ ] 5

b. Please provide a brief rationale for your clarity rating. What aspects of the matrix are particularly clear or unclear?

(Please type your answer here)

4.2. Utility of the Decision Model

a. On a scale of 1 (Not at all Useful) to 5 (Extremely Useful), how useful do you believe this Decision Matrix would be for a software architect when designing a new FM-based agent?

[ ] 1 [ ] 2 [ ] 3 [ ] 4 [ ] 5

b. In what specific design scenarios do you see this matrix being most or least useful? For instance, are there certain types of agent systems or application domains where you believe this matrix would be less applicable?

(Please type your answer here)

4.3. Practical Application Scenario

You are designing a system with a team of AI Analyst Agents to process thousands of online customer reviews for a new product. The core task for each agent is to use a Foundation Model to read a review, understand its sentiment, and categorize the feedback (e.g., "positive feedback on battery life," "negative feedback on screen brightness"). The system has two high-priority requirements:

- Reliability: The system must be highly fault tolerant. The failure of any single agent must not halt the overall analysis process; other agents must continue to operate and complete the task.
- Maintainability: The system's overall state must be easy to understand, monitor, and debug. A developer should be able to quickly diagnose issues (like a stalled or failed agent) without the complexity of inspecting every agent individually.

Based on the Decision Model, which Agent Coordination option (e.g., Centralized, Federated, Fully Distributed) presents the most challenging trade-off for this scenario, and why? Please briefly explain how the matrix helped your reasoning.

(Please type your answer here)

4.4. Suggestions for Improvement

Do you have any suggestions for improving the clarity or practical utility of the Decision Matrix?

(Please type your answer here)

Part 5: Concluding Remarks

5.1. General Feedback

Do you have any other comments, suggestions, or feedback on this research?

(Please type your answer here)

Thank you once again for your invaluable contribution.

Table 4: Decision Matrix: Impact of Design Options on Non-functional Qualities

Design Option	Perf.	Rel.	Rob.	Resil.	Sec.	Maint.	RAI	Usab.	Comp.
Single-modality	++	+	o	o	o	++	o	-	o
Multi-modality	–	-	+	+	-	–	+	++	-
Interactional Context	-	o	o	o	o	-	-	+	o
Environmental Context	o	+	o	o	o	o	o	+	o
Semantic Context	o	++	o	o	o	o	+	++	o
Passive Suggestion	+	o	+	o	o	o	+	o	-
Proactive Suggestion	-	+	-	+	o	-	-	-	++
Agent-as-a-coordinator	-	-	o	-	+	+	+	o	o
Agent-as-a-worker	+	+	o	o	o	++	o	o	o
Task Completion	+	o	o	o	o	+	o	+	o
Communication	-	-	+	+	–	-	++	++	++
Learning	-	–	++	++	-	–	–	+	o
Internal Planner	+	o	o	o	o	o	+	o	o
External Planner	-	++	++	o	o	+	-	o	o
Hybrid Approach	o	+	+	o	o	-	o	o	+
Internal Tool	o	o	-	o	-	-	-	o	o
External Tool	-	++	++	o	o	+	-	o	o
Single-path Plan Generator	+	-	-	-	o	++	+	-	o
Multi-path Plan Generator	-	o	++	++	o	-	o	++	o
Long-term Memory	o	+	o	+	o	o	-	+	+
Short-term Memory	++	o	+	o	o	-	++	o	++
Selective Forgetting	-	+	+	o	o	+	-	++	o
Natural Language Memory	-	o	o	o	o	o	++	++	o
Embedding Memory	++	o	o	o	o	-	-	-	o
Databases	o	++	++	++	++	++	+	+	o
Structured Lists	++	+	++	+	o	o	++	+	o
Memory Reading	+	–	o	o	o	o	o	+	o
Memory Writing	+	–	o	o	o	o	o	o	o
Memory Reflection	-	++	+	o	o	o	-	+	o
Memory Sharing	-	+	-	o	o	-	-	+	o
Centralized	+	-	-	-	+	+	+	o	o
Federated	o	+	o	o	o	-	o	o	o
Fully Distributed	-	++	++	++	-	–	–	o	o
Full Transparency	+	+	o	-	o	-	o	+	o
Partial Transparency	o	o	o	o	o	+	o	o	o
Configuration-free	++	o	+	o	o	o	++	o	++
Customizable	-	+	o	o	o	o	-	o	-
API	++	++	+	+	+	+	+	o	++
UI Understanding	–	o	o	o	o	o	o	++	–
API-based Learning	++	+	o	o	o	+	o	-	++
UI-based Learning	–	–	–	–	-	–	-	++	++
Inline Integration	+	o	o	o	o	o	+	o	o
Multi-source Integration	-	+	o	o	o	o	-	o	o
Physical Actuation	o	o	–	o	o	–	o	–	o
Virtual Actuation	o	+	o	o	o	+	o	o	o
Workflow/Plan Generation	o	o	o	o	o	o	+	o	o
Intermediate Result	o	o	o	o	o	o	o	o	o
Final Output	–	o	o	o	o	o	++	++	o
Self-reflection	-	+	+	+	o	-	+	o	o
Cross-reflection	-	o	o	o	-	-	+	o	o
Human Reflection	–	++	++	++	+	o	++	o	o
Environmental Reflection	+	+	o	o	o	+	+	o	o
Chain of Thought (CoT)	-	+	o	o	o	+	++	o	o
Tree of Thoughts (ToT)	–	o	o	o	o	–	o	o	o
Self-learning	o	o	o	-	o	-	-	o	o
In-context Learning	++	-	-	o	–	++	+	++	o
Group Learning	o	o	o	o	o	o	+	o	o
Centralized Workflow	+	o	-	o	o	o	+	o	o
Decentralized Workflow	-	+	++	++	++	-	–	o	o
Domain-specific AI	++	+	o	o	o	o	o	-	–
General Purpose AI	-	o	+	o	o	o	o	++	++
Single Model	o	o	-	o	-	o	+	-	o
Multi-model	–	++	+	+	o	–	+	o	-
Sovereign AI	o/-	++	+	o	++	+	++	o	–
Partially Sourced AI	+	o	o	o	+	o	+	o	+
Fully Externally Sourced AI	++	–	-	–	–	–	–	o	++
Centralized Training	–	o	o	o	+	+	-	o	o
Collaborative Learning	-	o	o	o	-	-	++	o	o/-

Note: '++' (strong positive), '+' (positive), 'o' (neutral/unspecified), '-' (negative), '–' (strong negative).
Perf.: Performance; **Rel.**: Reliability; **Rob.**: Robustness; **Resil.**: Resilience; **Sec.**: Security; **Maint.**: Maintainability; **RAI**: Responsible & Trustworthy AI; **Usab.**: Usability; **Comp.**: Compatibility.

Table 5: Mapping Table – Taxonomy Branches to Literature References

Taxonomy Branches	**References**
Input Modality > Single-modality	[32, 33]
Input Modality > Multi-modality	[34, 35, 36]
Context > Context Types > Interactional Context	[37]
Context > Context Types > Environmental Context	[38]
Context > Context Types > Semantic Context	[39]
Context > Goal Seeking > Passive Suggestion	[35]
Context > Goal Seeking > Proactive Suggestion	[40]
Role-playing > Agent-as-a-coordinator	[1, 5, 10, 41]
Role-playing > Agent-as-a-worker	[34, 42, 43, 44, 45, 46]
Goal Type > Task Completion	[4, 47, 48]
Goal Type > Communication	[48]
Goal Type > Learning	[4]
Planning > Planning Engine > Internal Planner	[51]
Planning > Planning Engine > External Planner	[52, 53]
Planning > Planning Engine > Hybrid Approach	[53]
Planning > Tool Selection > Internal	[4, 54]
Planning > Tool Selection > External	[54]
Planning > Plan Generation > Single-path Plan Generator	[56, 57]
Planning > Plan Generation > Multi-path Plan Generator	[51, 58, 59, 60]
Memory > Memory Types > Long-term Memory	[57, 61]
Memory > Memory Types > Short-term Memory	[62]
Memory > Memory Types > Selective Forgetting	[62]
Memory > Memory Format > Natural Language Memory	[4, 63]
Memory > Memory Format > Embedding Memory	[64]
Memory > Memory Format > Databases	[48]
Memory > Memory Format > Structured Lists	[39, 62]
Memory > Memory Operation > Memory Reading	[42, 43, 46, 62]
Memory > Memory Operation > Memory Writing	[65]
Memory > Memory Operation > Memory Reflection	[62]
Memory > Memory Operation > Memory Sharing	[12, 61, 66, 67]
Action > Agent Coordination > Centralized Coordination	[12, 16]
Action > Agent Coordination > Federated Coordination	[5, 42, 43, 46, 68]
Action > Agent Coordination > Fully Distributed Coordination	[16, 47, 62]
Action > Agent Communication > Full Transparency	[57, 58, 67, 69]
Action > Agent Communication > Partial Transparency > Goal-based Sharing	[16]
Action > Agent Communication > Partial Transparency > Role-based Sharing	[16]
Action > Agent Communication > Partial Transparency > Sensitive Data Withholding	[69, 70]
Action > Agent Communication > Partial Transparency > Context-aware Sharing	[70, 71, 72]
Action > Tool Calling > Tool Invocation > Configuration-free	[73]
Action > Tool Calling > Tool Invocation > Customizable	[73, 74, 75]
Action > Tool Calling > Tool Interface > API	[17, 54, 73, 76, 77, 78]
Action > Tool Calling > Tool Interface > UI Understanding	[74, 79, 80]
Action > Tool Calling > Tool User Type > User Experience Driven	[81, 82]
Action > Tool Calling > Tool User Type > Agent Experience Driven	[50]
Action > Tool Calling > Tool Learning > API-based Learning	[54, 73, 77]
Action > Tool Calling > Tool Learning > UI-based Learning	[35, 74]
Action > Tool Calling > Output Integration > Inline Integration	[54, 73]
Action > Tool Calling > Output Integration > Multi-source Integration	[76]
Action > Actuation > Physical Actuation	[38, 84, 85]
Action > Actuation > Virtual Actuation	[74, 78, 86, 87]
Reflection > Artifacts > Workflow/Plan Generation	[88]
Reflection > Artifacts > Intermediate Result	[44]
Reflection > Artifacts > Final Output	[44]
Reflection > Provider > Self-reflection	[4, 34, 41, 43, 44, 63, 89]
Reflection > Provider > Cross-reflection	[58, 63]
Reflection > Provider > Human Reflection	[52, 68, 85]
Reflection > Provider > Environmental Reflection	[4, 41, 46, 49, 78, 90]
Reflection > Mechanisms > Chain of Thought (CoT)	[59, 77, 91]
Reflection > Mechanisms > Tree of Thoughts (ToT)	[51, 60]
Learning Capability > Self-learning	[61, 63, 92, 93]
Learning Capability > In-context Learning	[51, 52, 57, 62, 67, 94]
Learning Capability > Group Learning > Peer-to-peer Learning	[67, 72]
Learning Capability > Group Learning > Hierarchical Learning	[58]
Workflow > Centralized Workflow	[12]
Workflow > Decentralized Workflow	[47]
Access to Underlying Models > Underlying Model Types > Domain-specific AI	[95]
Access to Underlying Models > Underlying Model Types > General Purpose AI	[32, 93]
Access to Underlying Models > Model Composition > Single Model	[16]
Access to Underlying Models > Model Composition > Multi-model	[38, 42, 58, 96, 97, 98, 99]
Access to Underlying Models > Model Sourcing > Sovereign AI	[32]
Access to Underlying Models > Model Sourcing > Partially Sourced AI	[95]
Access to Underlying Models > Model Sourcing > Fully Externally Sourced AI	[77]
Access to Underlying Models > Training Method > Centralized Training	[32, 78, 86]
Access to Underlying Models > Training Method > Collaborative Learning	[67, 100]